Microsoft

GitHub Actions 实战



© 版权所有 Microsoft Corporation。保留所有权利。

课程导览与安排



© 版权所有 Microsoft Corporation。保留所有权利。

概念迁移：从 GitLab CI 到 GitHub Actions

- 流水线 Pipeline → 工作流 Workflow
- 作业 Job → 作业 Job（通过 needs 编排依赖关系）
- 阶段 Stage → 作业依赖 / 矩阵 Matrix 策略
- Runner → GitHub 托管运行器 / 自托管运行器（含 ARC）
- 变量 Variables → 环境变量 env / Secrets / Contexts
- 制品 Artifacts → Artifacts / Cache（actions/cache）
- 受保护分支与规则 → Rulesets / 部署环境 Environment Protection（审批与门禁）

© 版权所有 Microsoft Corporation。保留所有权利。

概念迁移：从 GitLab CI 到 GitHub Actions

GitHub Actions

事件触发 + Job 依赖
Action 组件生态强
开源与跨平台体验突出

GitLab CI/CD

Pipeline + Stage 为主线
needs 可形成 DAG
一体化企业 DevSecOps 强

共同能力

Runner、缓存、产物、环境
矩阵、密钥、容器、
Kubernetes
自托管与云平台集成

不要只比较 YAML 语法。真正的成本来自权限治理、复用体系、Runner 运维和组织迁移。

© 版权所有 Microsoft Corporation。保留所有权利。

概念迁移：从 GitLab CI 到 GitHub Actions

GitHub Actions

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
```

GitLab CI/CD

```yaml
stages: [test]

test:
  stage: test
  image: node:22
  script:
    - npm ci
    - npm test
  rules:
    - if: $CI_COMMIT_BRANCH
```

能力	GitHub Actions	GitLab CI/CD
流水线何时创建	on: push / pull_request / schedule / workflow_dispatch	workflow: rules
Job 是否运行	jobs.<id>.if / step.if	rules / when
依赖关系	needs	needs
取消重复运行	concurrency + cancel-in-progress	interruptible + resource_group
人工批准	environment protection rules	manual job / protected environment

©

概念迁移：从 GitLab CI 到 GitHub Actions

GitHub 复用层级

Action：单个步骤能力
Reusable workflow：复用整组 Jobs
Composite Action：封装多步骤
Marketplace：快速组合第三方生态

GitLab 复用层级

include：导入外部 YAML
extends：继承公共配置
Component / Template：标准化组件
Parent-child / multi-project pipeline：跨项目编排



治理重点：第三方组件应锁定版本或提交哈希，并限制令牌权限、密钥暴露和供应链风险。

© 版权所有 Microsoft Corporation。保留所有权利。

课程目录

- 模块 1 认识 GitHub Actions
- 模块 2 构建 CI 工作流
- 模块 3自定义 Action 与可复用组件
- 模块 4 企业级管理与自托管运行器
- 模块 5 安全集成扩展：CodeQL
- Copilot 2026 新特性

© 版权所有 Microsoft Corporation。保留所有权利。

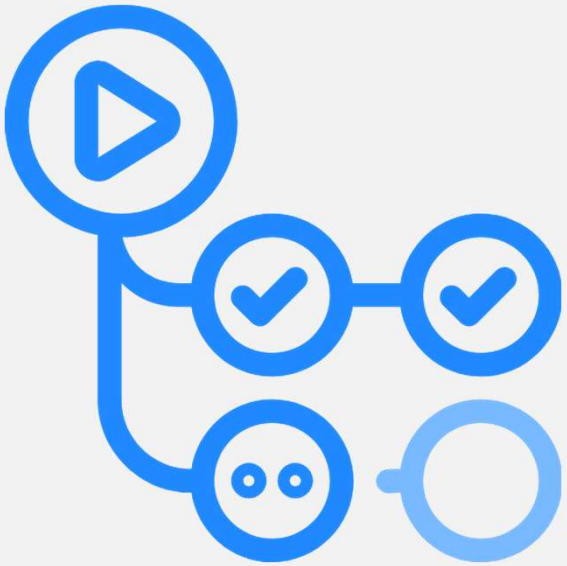
模块 1：认识 GitHub Actions



© 版权所有 Microsoft Corporation。保留所有权利。

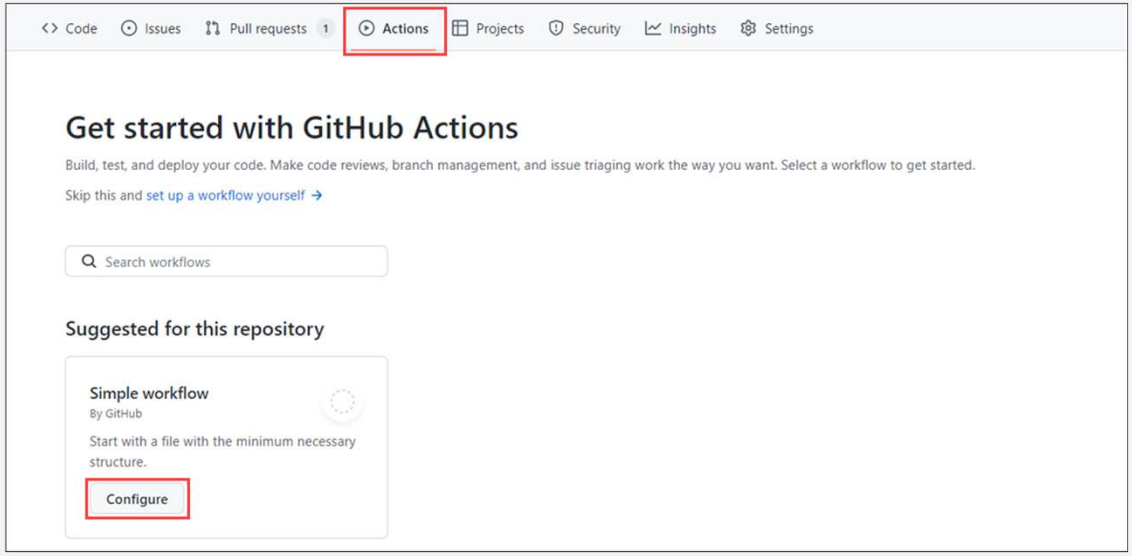
什么是 GitHub Actions?

GitHub Actions 是打包的脚本，用于在 GitHub 中自动执行软件开发工作流程中的任务。您可以配置 GitHub Actions 来触发满足您组织需求的复杂工作流程。触发器可以在开发人员每次将新源代码签入特定分支时触发，也可以按定时间隔触发，或者手动触发。最终形成可靠且可持续的自动化工作流程，从而显著缩短开发时间。



© 版权所有 Microsoft Corporation。保留所有权利。

在哪里可以找到 GitHub Actions?



GitHub Actions 是遵循yml数据格式的脚本。每个仓库都有一个**“Actions”**选项卡，可让您快速轻松地开始设置您的第一个脚本。

如果您看到一个您认为可能是一个很好的起点的工作流程，只需选择**配置**按钮来添加脚本并开始编辑源yml 。

© 版权所有 Microsoft Corporation。保留所有权利。

在哪里可以找到 GitHub Actions？ （续）

但是，除了“操作”选项卡上显示的 GitHub 操作之外，您还可以：



GitHub 市场

GitHub Marketplace中搜索 GitHub Actions。GitHub Marketplace 允许您发现和购买扩展工作流程的工具。



开源项目

搜索开源项目。例如，GitHub Actions 该组织拥有许多流行的开源存储库，其中包含您可以使用的 GitHub Actions。



从零开始

从头开始编写你自己的 GitHub Actions。你可以将它们开源，甚至可以将它们发布到 GitHub Marketplace。

© 版权所有 Microsoft Corporation。保留所有权利。

使用开源 GitHub Actions

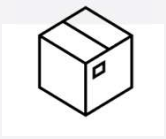
许多 GitHub Actions 都是开源的，任何人都可以使用。但是，与任何开源软件一样，在项目中使用它们之前，您需要仔细检查它们。与开源软件的推荐社区标准（例如包含 README、行为准则、贡献文件和问题模板）类似，您在使用 GitHub Actions 时可以遵循以下建议：

- 检查操作的`action.yml`文件中的输入、输出，并确保代码按照其说明执行。
- 检查操作是否在 GitHub Marketplace 中。即使操作不一定非要在 GitHub Marketplace 中才有效，这仍然是一个很好的检查。
- 检查该操作是否已在 GitHub Marketplace 中验证。这意味着 GitHub 已批准使用此操作。但是，在使用前您仍应进行审核。
- 通过指定 Git ref、SHA 或标签来包含您正在使用的操作的版本。

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub 操作的类型

有**三种**类型：容器操作、JavaScript 操作和复合操作。



对于**容器操作**，环境是操作代码的一部分。这些操作只能在 GitHub 托管的 Linux 环境中运行。容器操作支持多种不同的语言。



JavaScript 操作不包含代码中的环境。您必须指定环境才能执行这些操作。您可以在云端或本地的 VM（虚拟机）中运行这些操作。JavaScript 操作支持 Linux、macOS 和 Windows 环境。



复合操作允许您在一个操作中组合多个工作流步骤。例如，您可以使用此功能将多个运行命令捆绑到一个操作中，然后创建一个使用该操作将捆绑的命令作为单个步骤执行的工作流。

© 版权所有 Microsoft Corporation。保留所有权利。

什么是 GitHub Actions 工作流程？

GitHub *Actions 工作流程*是您在仓库中设置的流程，用于自动执行软件开发生命周期任务，包括 GitHub Actions。通过工作流程，您可以在 GitHub 上构建、测试、打包、发布和部署任何项目。

要创建工作流程，您需要将操作添加到GitHub 存储库中`.github /workflows`目录中的.yml文件中。

在接下来练习中，您的工作流文件*main.yml*将如下所示：

注意`on:`属性。这是一个*触发器*，用于指定此工作流何时运行。这里，当仓库发生推送事件时，它会触发运行。

```yaml
name: A workflow for my Hello
World file
on: push
jobs:
  build:
    name: Hello world action
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v1
    - uses: ./action-a
      with:
        MY_NAME: "Mona"
```

© 版权所有 Microsoft Corporation。保留所有权利。

什么是 GitHub Actions 工作流程？ （续）

您可以指定单个事件（例如`on: push`）、事件数组（例如`on: [push, pull_request]`），或者指定一个事件配置映射，用于调度工作流或将工作流的执行限制在特定的文件、标签或分支更改中。该映射可能如下所示：

除非您指定类型，否则事件将在其所有活动类型上触发。有关事件及其活动类型的完整列表，请参阅：触发工作流的事件 在 GitHub 文档中。

```yaml
on:
  # Trigger the workflow on push or pull request,
  # but only for the main branch
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  # Also trigger on page_build, as well as release
created events
  page_build:
  release:
    types: # This configuration doesn't affect the
page_build event above
      - created
```

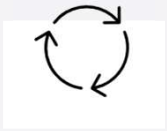
© 版权所有 Microsoft Corporation。保留所有权利。

工作流程必须至少包含一个*作业*

*运行器(runner)*关联的工作流程的一部分。运行器可以是 GitHub 托管的，也可以是自托管的，作业可以在机器或容器中运行。



`runs-on:`属性指定运行器。在这里，您告诉工作流在`ubuntu-latest`上运行此作业。



每个作业都有要完成的步骤。在我们的示例中，该步骤使用操作 actions/checkout@v1 来检出存储库。值得注意的是`uses: ./action-a`值，它是您在action.yml文件中构建的容器操作的路径。



此工作流文件的最后一部分设置了此工作流的`MY_NAME`变量值。回想一下，容器操作采用了一个名为`MY_NAME`的输入。

有关工作流程语法的更多信息，请参阅GitHub Actions 的工作流程语法。

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub 托管运行器与自托管运行器

说到运行器，有两种选择：GitHub 托管的运行器或自托管的运行器。如果您使用 GitHub 托管的运行器，则每个作业都会在虚拟环境的全新实例中运行。您定义的 GitHub 托管运行器类型 `runs-on: {operating system-version}` 指定了该环境。对于自托管的运行器，您需要应用 self-hosted 标签、其操作系统和系统架构。例如，具有 Linux 操作系统和 ARM32 架构的自托管运行器如下所示： `runs-on: [self-hosted, linux , ARM32]。`

每种类型的运行器都有其优势，但 GitHub 托管的运行器虽然选项有限，但却能更快速、更简单地运行您的工作流程。自托管运行器是一种高度可配置的方式，可在您自己的自定义本地环境中运行工作流程。您可以在本地或云端运行自托管运行器。您还可以使用自托管运行器创建具有更强处理能力或内存的自定义硬件配置。这将有助于运行更大规模的作业、安装本地网络上可用的软件，以及选择 GitHub 托管运行器未提供的操作系统。

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub Actions 可能有使用限制



GitHub Actions 有一些使用限制，具体取决于您的 GitHub 套餐以及您的运行器是 GitHub 托管还是自托管。有关使用限制的更多信息，请参阅使用限制、计费和管理 在 GitHub 文档中。

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub 托管了更大的运行器

GitHub 为需要更多资源的工作流程提供更强大的运行器。这些运行器由 GitHub 托管，与标准运行器相比，可提供更大的 CPU、内存和磁盘空间。它们旨在高效处理资源密集型工作流程，确保高要求任务的最佳性能。

流道尺寸和标签

更大型的运行器提供多种配置，提供增强型 vCPU、RAM 和 SSD 存储，以满足多样化的工作流程需求。这些配置非常适合以下场景：

- 使用大量源文件编译大型代码库。
- 运行全面的测试套件，包括集成和端到端测试。
- 处理大型数据集以进行数据分析或机器学习任务。
- 构建具有复杂依赖关系或大型二进制输出的应用程序。
- 执行高性能模拟或计算建模。
- 执行视频编码、渲染或其他多媒体处理工作流程。

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub 托管了更大的运行器（续）

`runs-on`属性中指定所需的运行器标签。例如，要使用具有 16 个 vCPU 和 64 GB RAM 的运行器，您需要设置`runs-on: ubuntu-latest-16core` 。

```yaml
jobs:
  build:
    runs-on: ubuntu-latest-16core
    steps:
      - uses: actions/checkout@v2
      - name: Build project
        run: make build
```

`ubuntu-latest运行器`相同的预安装工具来保持与现有工作流程的兼容性。

有关较大跑步者的跑步者尺寸的更多详细信息，请参阅GitHub 文档。

© 版权所有 Microsoft Corporation。保留所有权利。

管理体型较大的跑步者

GitHub 提供了有效管理大型运行器的工具，确保最佳的资源利用率和成本管理。以下是管理大型运行器的一些关键方面：

监控使用情况

您可以通过仓库或组织设置中的 GitHub Actions 使用情况页面监控大型运行器的使用情况。此页面提供有关运行的作业数量、总运行时间以及相关成本的洞察。

管理访问

要控制对大型运行器的访问，您可以配置仓库或组织级别的策略。这可确保只有授权的工作流程或团队才能使用这些高资源运行器。

成本管理

大型跑步机将根据其使用情况产生额外费用。为了管理成本，请考虑以下事项：

- 仅对需要大量资源的工作流程使用较大的运行器。
- 优化工作流程以减少运行时间。
- 定期监控账单详情以跟踪费用。

扩展工作流程

如果您的工作流程需要频繁使用更大的运行器，请考虑以下扩展策略：

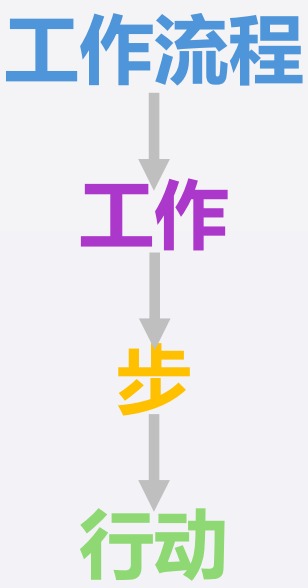
- 使用自托管运行器来实现可预测的工作负载。
- 将工作流程拆分成更小的作业，以在标准运行器之间分配负载。

© 版权所有 Microsoft Corporation。保留所有权利。

识别 GitHub Actions 的组件

GitHub Actions 的组件

GitHub Actions 工作流程中有几个组件可以协同运行任务或作业。简而言之，一个事件触发包含*作业的工作流程*。然后，该作业使用*步骤*来指示将在工作流程中运行哪些*操作*。




```
name: A workflow for my Hello World file
on: push

jobs:
  build:
   name: Hello world action
   runs-on: ubuntu-latest

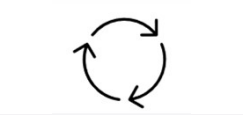
   steps:

      - uses: actions/checkout@v2
      - uses: ./action-a
      with:
         MY_NAME: "Mona"
```

© 版权所有 Microsoft Corporation。保留所有权利。

GitHub Actions 的组件

为了更好地了解这些组件如何协同工作，让我们快速看一下每一个组件。



工作流程

工作流程是您添加到仓库的自动化流程。一个工作流程至少需要包含一个作业，并且不同的事件可以触发它。您可以使用它在 GitHub 上构建、测试、打包、发布或部署仓库的项目。



工作

作业是工作流程中的第一个主要组件。作业是工作流程的一部分，它将与运行器关联。运行器可以是 GitHub 托管的，也可以是自托管的，并且作业可以在机器或容器中运行。您可以使用`runs-on ：属性指定运行器。在这里，您告诉工作流程在ubuntu-latest`上运行此作业。我们将在下一单元中详细讨论运行器。

© 版权所有 Microsoft Corporation。保留所有权利。

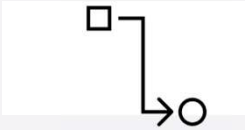
GitHub Actions 的组件

为了更好地了解这些组件如何协同工作，让我们快速看一下每一个组件。



步骤

步骤是可以在作业中运行命令的独立任务。在我们前面的示例中，该步骤使用操作 `actions/checkout@v2`来检出存储库。值得关注的是`uses: ./action-a`值。这是您将在`action.yml`文件中构建的容器操作的路径。



行动

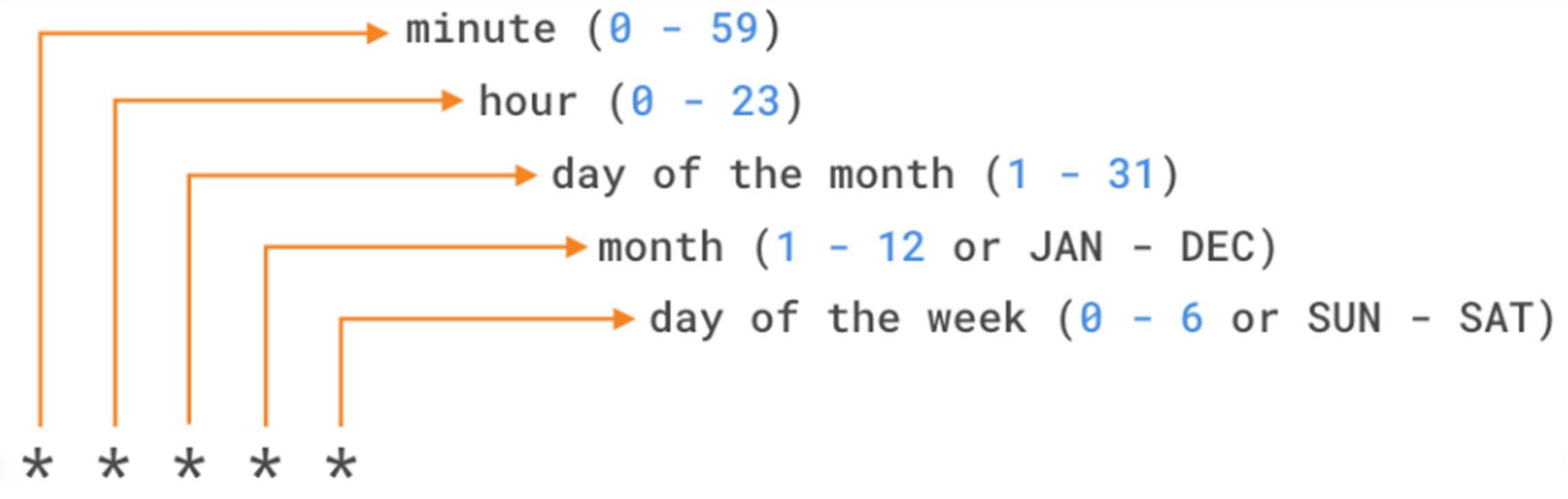
工作流程中的操作是独立执行的命令。这些独立命令可以引用 GitHub 操作（例如使用您自己的自定义操作），也可以引用社区操作（例如我们在上例中使用的`actions/checkout@v2` ）。您还可以运行诸如`run: npm install -g bats`之类的命令，在运行器上执行命令。

© 版权所有 Microsoft Corporation。保留所有权利。

配置工作流程以针对计划事件运行

如前所述，您可以将工作流程配置为在 GitHub 上发生特定活动时、GitHub 外部发生事件时或在预定时间运行。schedule事件允许您使用POSIX cron 语法触发工作流程在特定的 UTC 时间运行。

这个cron语法有五个*字段，每个字段代表一个时间单位。



© 版权所有 Microsoft Corporation。保留所有权利。

预定事件示例

例如，如果你想每 15 分钟运行一次工作流，则计划 事件看起来如下：

```yaml
on:
  schedule:
    - cron:  '*/15 * * * *'
```

如果您希望每周日凌晨 3:00 运行工作流，则计划事件将如下所示：

```yaml
on:
  schedule:
    - cron:  '0 3 * * SUN'
```

您还可以使用运算符指定值范围或调用计划工作流。运行计划工作流的最短间隔为每五分钟一次，并且它们在默认分支或基础分支的最新提交上运行。

© 版权所有 Microsoft Corporation。保留所有权利。

配置工作流程以运行手动事件（续）

除了`workflow_dispatch`之外，您还可以使用 GitHub API 触发名为`repository_dispatch`的 Webhook 事件。此事件允许您为 GitHub 之外发生的活动触发工作流程。它本质上是向您的仓库发送 HTTP 请求，要求 GitHub 通过某个操作或 Webhook 触发工作流程。

使用此手动事件需要您做两件事：

1. 向 GitHub 端点`/repos/{owner}/{repo}/dispatches`发送一个`POST`请求，并在请求正文中包含 webhook 事件名称。
2. 配置您的工作流程以使用`repository_dispatch`事件。

```
curl \
  -X POST \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/repos/octocat/hello-
world/dispatches \
  -d '{"event_type":"event_type"}'
```

```
on:
  repository_dispatch:
    types: [opened, deleted]
```

© 版权所有 Microsoft Corporation。保留所有权利。

配置要针对 webhook 事件运行的工作流

最后，您可以配置一个工作流程，使其在 GitHub 上发生特定 Webhook 事件时运行。大多数 Webhook 事件都可以从多个活动中触发。如果某个 Webhook 有多个活动，您可以指定一个活动类型来触发工作流程。

例如，您可以为`check_run`事件运行工作流，但只能为`requested`或`requested_action`运行工作流活动类型。

```yaml
on:
  check_run:
    types: [rerequested, requested_action]
```

© 版权所有 Microsoft Corporation。保留所有权利。

配置工作流程以针对手动事件运行

除了计划事件之外，您还可以使用`workflow_dispatch`事件手动触发工作流程。此事件允许您使用 GitHub REST API 或通过选择GitHub 仓库中**“操作”选项卡中的“运行工作流程”**按钮来运行工作流程。使用`workflow_dispatch` ，您可以选择要在哪个分支上运行工作流程，以及设置GitHub 将在 UI 中以表单元素形式呈现的可选输入。

```yaml
on:
  workflow_dispatch:
    inputs:
      logLevel:
        description: 'Log level'
        required: true
        default: 'warning'
      tags:
        description: 'Test scenario tags'
```

© 版权所有 Microsoft Corporation。保留所有权利。

Repository_dispatch

`repository_dispatch`是 GitHub Actions 中的一个自定义事件，允许外部系统（甚至其他 GitHub 工作流程）通过向 GitHub API 发送 POST 请求来手动触发工作流程。它支持灵活的自动化，并与需要在仓库中启动工作流程的外部工具、脚本或系统集成。

用例

- 从外部 CI/CD 工具触发工作流程。
- 协调多 repo 部署（例如，Repo A 完成构建→触发 Repo B）。
- 根据外部事件（webhook、监控警报、GitHub 之外的 CRON 作业）启动自动化。
- 在存储库之间或monorepos内链接工作流执行。

© 版权所有 Microsoft Corporation。保留所有权利。

监听repository_dispatch 的示例工作流

```yaml
name: Custom Dispatch Listener

on:
  repository_dispatch:
    types: [run-tests, deploy-to-prod]  # Optional filtering

jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - name: Echo the payload
        run: |
          echo "Event type: ${{ github.event.action }}"
          echo "Payload value: ${{ github.event.client_payload.env }}"
```

关键要素：

- **类型**：可选。定义自定义事件类型，例如运行测试、部署到生产环境等。
- github.event.client_payload： 访问调度事件中传递的任何其他自定义数据。
- github.event.action： **发送的**event_type的名称。

© 版权所有 Microsoft Corporation。保留所有权利。

通过 API 触发事件

您必须向 GitHub REST API v3 端点发送 POST 请求：

```
POST https://api.github.com/repos/OWNER/REPO/dispatches
```

授权

- 需要具有 repo 范围的个人访问令牌 (PAT)。
- 对于组织，请确保您的令牌的访问设置正确。

示例命令结构

```
curl -X POST \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: token YOUR_GITHUB_TOKEN" \
  https://api.github.com/repos/OWNER/REPO/dispatches \
  -d '{"event_type":"run-tests","client_payload":{"env":"staging"}}'
```

© 版权所有 Microsoft Corporation。保留所有权利。

有效载荷结构

```
{
  "event_type": "run-tests",
  "client_payload": {
    "env": "staging"
  }
}
```

参数

场地	类型	描述	必需的
事件类型	细绳	事件的自定义名称。这将映射到工作流触发器中的类型值	是的
客户端有效载荷	目的	任意 JSON 有效负载，用于将自定义数据发送到工作流（github.event.client_payload）	不
不			

© 版权所有 Microsoft Corporation。保留所有权利。

Repository_dispatch参数解析

向 GitHub API 端点发出 POST 请求时，必须传递包含两个主要参数的 JSON 主体：

事件类型

这是您定义的必需自定义字符串，GitHub 将视其为调度的“操作”或“类型”。它用于识别触发工作流的原因，并过滤正在监听特定类型的工作流。

- 格式：
  - 类型：字符串
  - 例如：“deploy”、“run-tests”、“sync- db ”、“build-docker”
- 在工作流中使用：用于监听特定事件类型并访问工作流内部的值。这有助于将单个工作流重用于多种用途，并使自动化更加有序、事件驱动。
- 例子：

```
- name: Print event type
  run: echo "Event type: ${{ github.event.action }}”
```

客户端有效载荷

这是一个自由格式的 JSON 对象，允许您在调度时发送自定义数据。您可以定义其结构，并在工作流中访问它。

- 格式：
  - 类型：对象
  - 自定义键和值
- 在工作流中使用：用于多环境部署、版本发布或从另一个系统或管道传递上下文，并支持参数化工作流，类似于输入参数。
- 例子：

```
- name: Show payload values
  run: |
    echo "Environment:
${{ github.event.client_payload.env }}"
    echo "Version:
${{ github.event.client_payload.version
}}"
```

© 版权所有 Microsoft Corporation。保留所有权利。

有效载荷分解示例

```
{
  "event_type": "deploy-to-prod",
  "client_payload": {
    "env": "production",
    "build_id": "build-456",
    "initiator": "admin_user",
    "services": ["web", "api", "worker"]
  }
}
```

© 版权所有 Microsoft Corporation。保留所有权利。

使用条件关键字

在工作流程文件中，您可以访问上下文信息并评估表达式。虽然表达式通常与工作流程文件中的条件`if`关键字一起使用来确定某个步骤是否应该运行，但您可以使用任何受支持的上下文和表达式来创建条件。重要的是要知道，在工作流程中使用条件时，需要使用特定的语法`${{ <expression> }}`。这会告诉 GitHub 评估表达式，而不是将其视为字符串。

例如，一个使用`if`条件判断`github.ref`（触发工作流程运行的分支或标签引用）是否与`refs/heads/main`匹配的工作流程。为了继续执行，工作流程应如下所示：

```yaml
name: CI
on: push
jobs:
  prod-check:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      ...
```

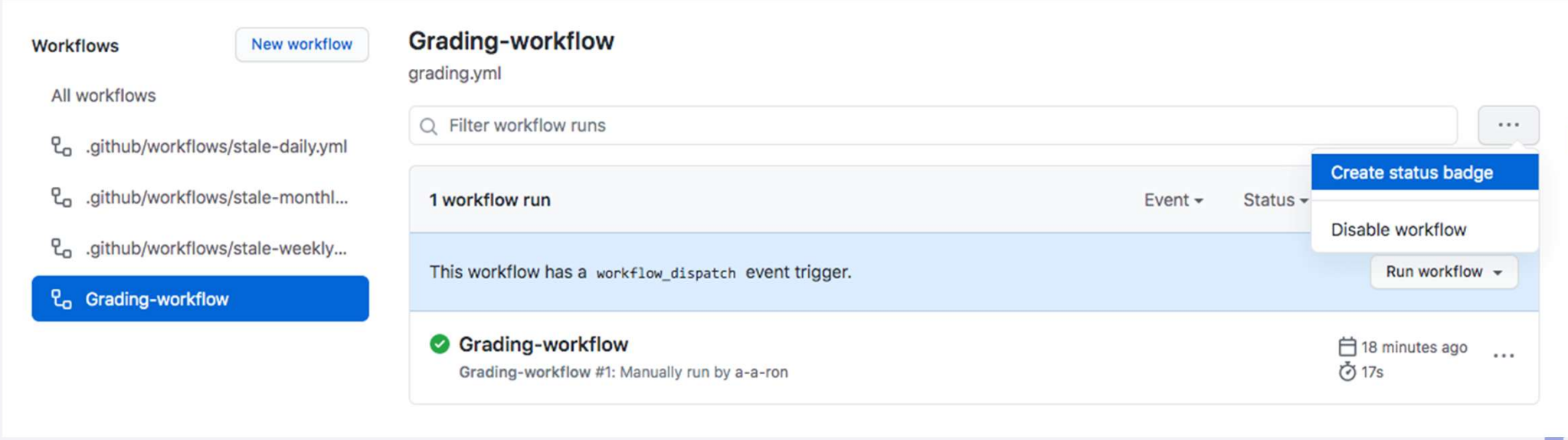
在此示例中，语法中缺少`${{ }}`。对于某些表达式，例如`if`条件语句，您可以省略表达式语法。GitHub 会自动评估其中一些常见表达式，但如果您忘记了 GitHub 会自动评估哪些表达式，您可以随时添加它们。

有关工作流语法和表达式的更多信息，请查看GitHub Actions 的工作流语法。

© 版权所有 Microsoft Corporation。保留所有权利。

禁用和删除工作流程

将工作流添加到仓库后，您可能会遇到需要暂时禁用该工作流的情况。您可以通过 GitHub 或 GitHub REST API 来停止触发该工作流，而无需从仓库中删除文件。当您希望再次启用该工作流时，可以使用相同的方法轻松完成。



© 版权所有 Microsoft Corporation。保留所有权利。

禁用和删除工作流程（续）

在以下某些情况下，禁用工作流可能会很有用：

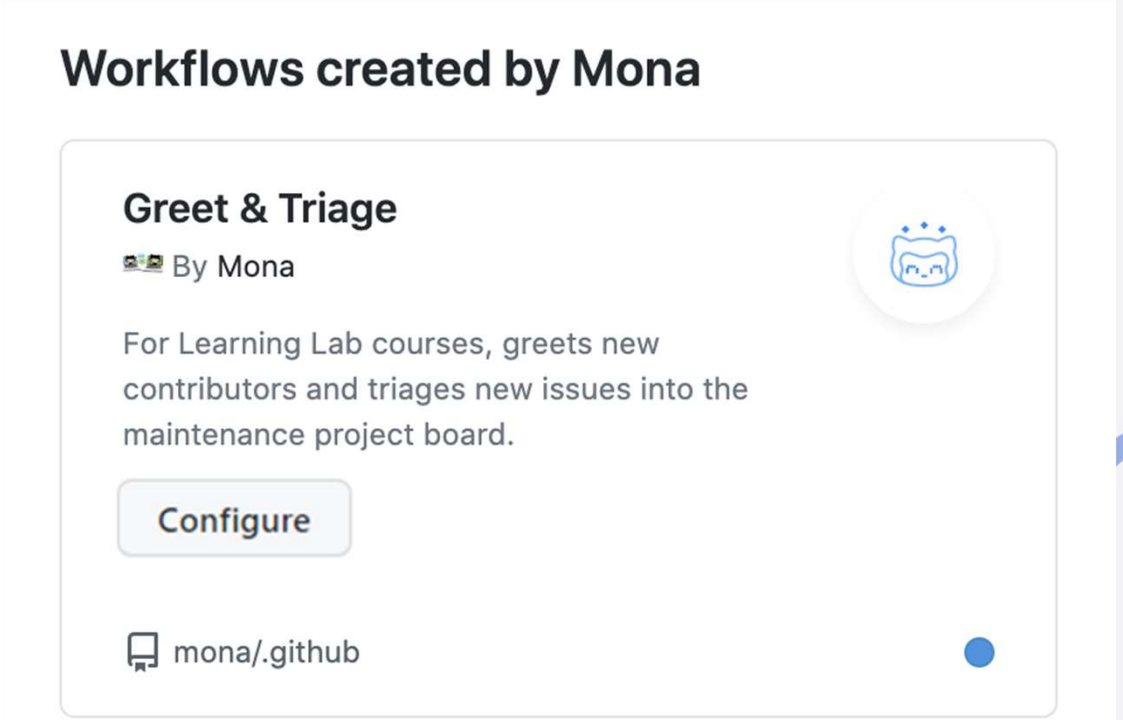
- 工作流程中的错误会产生过多或错误的请求，从而对外部服务产生负面影响。
- 您想要暂时暂停不重要且占用您帐户太多时间的工作流程。
- 您想要暂停向已关闭的服务发送请求的工作流。
- 您正在处理一个分支，并且不需要它所包含的一些工作流程的所有功能（例如计划的工作流程）。

`DELETE /repos/{owner}/{repo}/actions/runs/{ run_id }`取消正在进行的工作流程运行。请记住，当您取消工作流程运行时，GitHub 将取消该运行中的所有作业和步骤。

© 版权所有 Microsoft Corporation。保留所有权利。

使用组织的模板化工作流程

如果您拥有一个供组织内多个团队使用的工作流程，则无需为每个仓库重新创建相同的工作流程。相反，您可以使用在组织的`.github`仓库中定义的工作流程模板来提高整个组织的一致性。组织内的任何成员都可以使用组织模板工作流程，并且该组织内的任何仓库都可以访问这些模板工作流程。



您可以通过以下方式找到这些工作流程：导航到组织内某个仓库的**“操作”选项卡，选择“新建工作流程”**，然后找到该组织的工作流模板部分，标题为“按*组织名称创建的工作流*”。例如，名为 Mona 的组织有一个如下所示的模板工作流。

© 版权所有 Microsoft Corporation。保留所有权利。

使用特定版本的动作

在工作流程中引用操作时，我们建议您引用该操作的特定版本，而不仅仅是操作本身。通过引用特定版本，您可以设置安全措施，防止推送到操作的意外更改可能破坏工作流程。您可以通过以下几种方式引用操作的特定版本：

```yaml
steps:
  # Reference a specific commit
  - uses: actions/setup-node@c46424eee26de4078d34105d3de3cc4992202b1e
  # Reference the major version of a release
  - uses: actions/setup-node@v1
  # Reference a minor version of a release
  - uses: actions/setup-node@v1.2
  # Reference a branch
  - uses: actions/setup-node@main
```

有些引用比其他引用更安全。例如，引用特定分支将基于该分支的最新更改运行该操作，而您可能希望或不希望这样做。通过引用特定的版本号或提交 SHA 哈希，您可以更明确地了解所运行操作的版本。为了提高稳定性和安全性，我们建议您在工作流程中使用已发布操作的提交 SHA。

© 版权所有 Microsoft Corporation。保留所有权利。

模块 2：构建持续集成 CI 工作流



© 版权所有 Microsoft Corporation。保留所有权利。

从模板创建工作流

要创建工作流程，请先从包含常用预配置作业和步骤的模板开始。如果您不熟悉工作流程、作业和步骤，请查看使用 GitHub Actions 自动执行开发任务 模块。

1

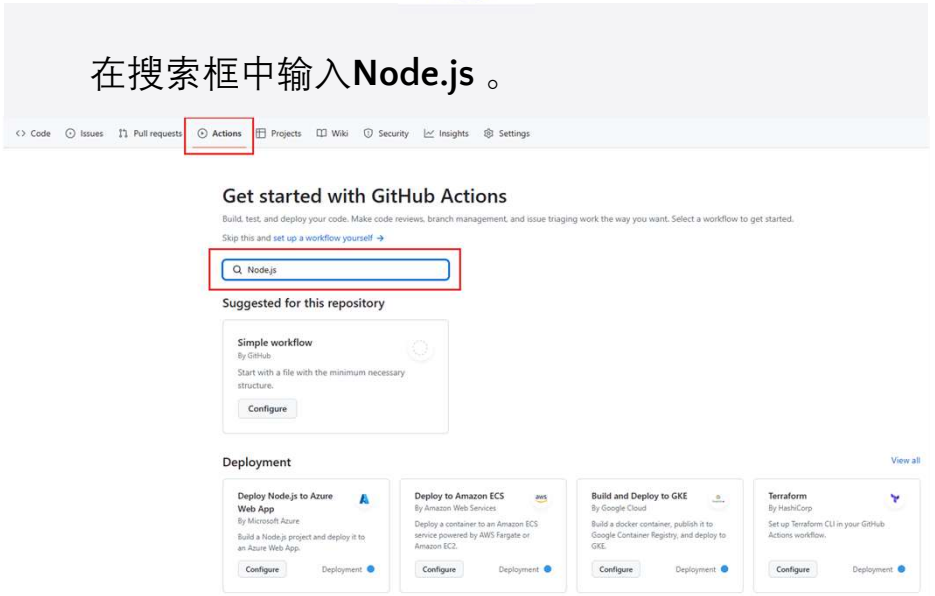
在存储库的主页上，选择**“操作”**选项卡，然后选择**“新建工作流程”**。

2

在**“选择工作流程”**页面上，您可以从许多不同的模板中进行选择。例如，Node.js 模板会全新安装 Node 依赖项、构建源代码并针对不同版本的 Node 运行测试。另一个示例是 Python 软件包模板，它会安装 Python 依赖项并针对不同版本的 Python 运行测试（包括 lint）。

3

在搜索框中输入**Node.js** 。

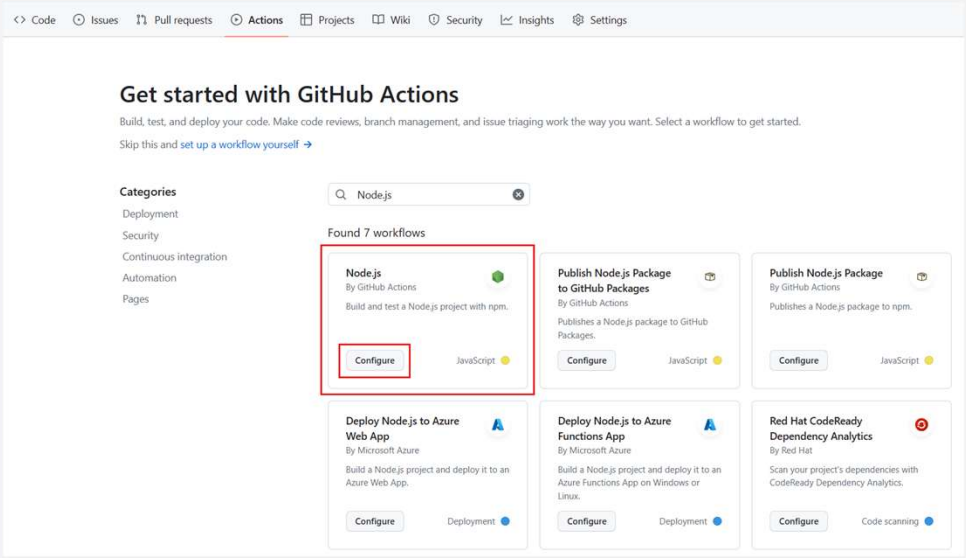


© 版权所有 Microsoft Corporation。保留所有权利。

从模板创建工作流程（续）

4

在搜索结果的 Node.js 窗格中，选择“**配置**”。



5

您可以在新创建的文件node.js.yml中看到这个默认的 Node.js 模板工作流程。

```yaml
name: Node.js CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:

    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [14.x, 16.x, 18.x]

    steps:
    - uses: actions/checkout@v3
    - name: Use Node.js ${{ matrix.node-version }}
      uses: actions/setup-node@v3
      with:
        node-version: ${{ matrix.node-version }}
        cache: 'npm'
    - run: npm ci
    - run: npm run build --if-present
    - run: npm test
```

© 版权所有 Microsoft Corporation。保留所有权利。

工作流程细分：关键属性解释

注意`on:`属性。此工作流在推送到存储库时触发，以及在主分支上发出拉取请求时触发。

有一项`作业`。让我们回顾一下它的作用。

`连续运行`： 属性指定，对于操作系统，工作流在`ubuntu-latest上运行`。node-version: 属性指定有三个版本，分别针对 Node 版本 14.x、16.x 和 18.x。稍后，当我们自定义工作流时，我们将深入描述`矩阵部分。`

作业中的`步骤`使用GitHub Actions actions/checkout@v3 将代码从存储库放入虚拟机的操作，以及 actions/setup-node@v3 设置正确的 Node.js 版本。我们指定要使用`${{ matrix.node-version }}测试三个版本的 Node.js` 属性。此属性引用了我们之前定义的矩阵。cache属性指定了默认目录中用于缓存的包管理器。

此步骤的最后一部分执行 Node.js 项目使用的命令。npm `ci`命令从*package-* `lock.json`*文件安装依赖项*，`npm run build --if-present`命令运行构建脚本（如果存在），`npm test` 命令运行测试框架。请注意，此模板在同一作业中同时包含构建和测试步骤。

© 版权所有 Microsoft Corporation。保留所有权利。

Npm文档

要了解有关npm 的更多信息，请查看npm文档：

- npm 安装
- npm 运行
- npm 测试

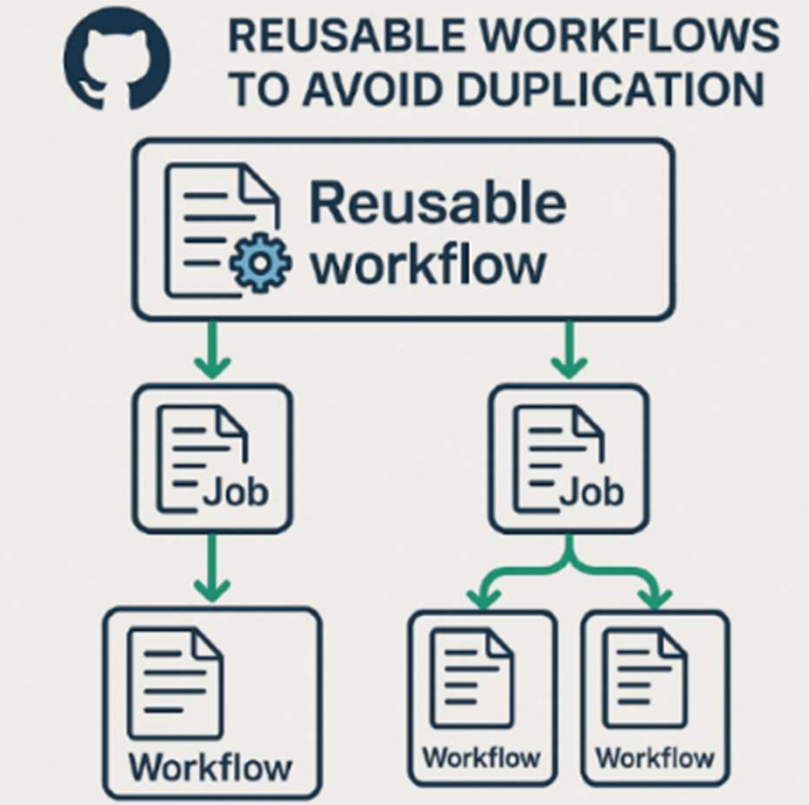
除了单个npm命令之外，团队还可以受益于可重复使用的工作流程，以简化和标准化重复的自动化步骤。

通过利用可重复使用的工作流程，您可以减少冗余、提高可维护性并确保整个 CI/CD 管道的一致性。

© 版权所有 Microsoft Corporation。保留所有权利。

如何利用可重复使用的工作流程来避免重复

随着团队规模和项目规模的扩大，我们常常会看到相同的步骤（例如代码签出、依赖项安装、测试和部署）在多个工作流文件中重复执行。这种重复不仅会使代码库变得混乱，还会在需要更改时增加维护时间。可重用工作流解决了这个问题，它允许您定义一次自动化逻辑，然后从其他工作流中调用它。可重用工作流是特殊的 GitHub Actions 工作流，可以被其他工作流调用，就像编程中的函数一样。您可以创建它们来共享重复的逻辑，例如构建步骤、测试过程或部署策略。创建后，您可以从同一仓库中的任何其他工作流甚至跨不同仓库引用它们。



© 版权所有 Microsoft Corporation。保留所有权利。

可重复使用的工作流程

为什么要使用它们?

- 一致性：团队可以在所有项目中遵循相同的自动化标准。
- 效率：您无需复制和粘贴步骤，只需指向可重复使用的工作流程。
- 易于更新：当流程发生变化（例如，新的测试步骤）时，您可以在一个地方进行更新，并且使用它的所有工作流程都会自动受益。
- 可扩展性：非常适合管理多项服务的平台或 DevOps 团队。

让我们探索如何使用可重复使用的工作流程来改进您的项目。

如何实现可重用的工作流

要利用可重复使用的工作流程：

- 在您的代码库文件夹中创建一个可重复使用的工作流。此文件将包含您想要共享的自动化步骤，例如测试、构建或部署。
- workflow_call事件进行配置来明确启用工作流，以便其可重复使用。
- 在您的主要工作流程（调用者工作流程）中，您可以引用这个可重复使用的文件并提供任何所需的输入或机密。

为了说明可重用工作流的优势，请考虑以下真实场景。

© 版权所有 Microsoft Corporation。保留所有权利。

进一步了解：可重复使用的工作流程

真实示例

假设您的组织有 10 个微服务，并且所有微服务都需要相同的步骤来：

- 运行测试
- Lint 代码
- 部署到特定环境

如果没有可复用的工作流，每个代码库都会包含重复的逻辑。有了可复用的工作流，您可以：

- 在中心文件中定义一次流程（例如 ci- standard.yml）
- 从每个服务自己的工作流中调用此文件，传递环境或应用程序名称等变量

现在，如果添加了新的安全步骤或工具（例如扫描漏洞），您只需在可重用的工作流中添加一次即可。所有 10 个服务将立即使用更新后的流程，而无需逐一进行修改。

最佳实践

- 如果您计划在团队之间共享可重复使用的工作流程，请将其集中在一个存储库中。
- 使用分支或标签来对您的工作流程进行版本控制（例如，使用@v1），这样更改就不会意外破坏所有内容。
- 清楚地记录输入和秘密——可重复使用的工作流程通常依赖于输入和秘密，团队需要知道提供什么。
- 如果您只需要重复使用几个步骤而不是完整的工作流程，请与复合操作结合使用。

概括

可重复使用的工作流是在任何工程团队中强制一致性、减少重复和扩展 DevOps 实践的有效方法。无论您管理的是单一仓库、微服务还是开源库，可重复使用的工作流都可以简化自动化流程，使 CI/CD 更快、更清晰、更易于管理。

© 版权所有 Microsoft Corporation。保留所有权利。

根据其影响识别触发工作流的事件

了解触发 GitHub Actions 工作流程的因素（无论是推送到分支、拉取请求、计划作业还是手动调度）对于调试、审核和改进 CI/CD 管道至关重要。您可以通过检查工作流程运行、仓库更改或所涉及的问题/拉取请求来识别触发事件。



什么是工作流触发器?

工作流程触发器是导致工作流程启动的事件。GitHub 支持各种类型的触发器，包括：

- `push`或`pull_request`（基于代码更改）
- `workflow_dispatch`（手动触发）
- 计划（cron作业）
- `repository_dispatch`（外部系统）
- 问题、讨论和 PR 事件（例如`issues.opened`、pull_request.closed）

© 版权所有 Microsoft Corporation。保留所有权利。

在哪里识别触发事件?

您可以通过多种方式识别触发事件:

从 GitHub Actions UI

- 导航到存储库中的“操作”选项卡。
- 单击工作流程运行。
- 事件类型（例如，`push`、`pull_request`、`workflow_dispatch`）显示在工作流运行摘要的顶部。

在日志或工作流中使用 github.event_name

- GitHub 会在工作流程运行期间公开上下文数据。github.event_name变量会告诉您触发该工作流程的事件。
- 您可以将其打印出来以进行调试:

```
-name: Show event trigger
  run: echo "Triggered by
${{ github.event_name }}"
```

使用工作流运行详细信息

- 如果您以编程方式检查工作流运行（例如，通过 API），则运行对象包含指定触发器的事件属性。
- 您还可以找到提交 SHA、参与者和时间戳来追踪触发的原因。

© 版权所有 Microsoft Corporation。保留所有权利。

从存储库效应推断触发器

有时，您可能无法直接访问工作流程运行，但希望根据仓库活动推断触发该运行的原因。具体方法如下：

观察到的行为	触发事件
新的提交被推送到`主目录`并运行工作流程	`推送`事件
新的拉取请求已打开或更新	`pull_request`事件
贡献者手动运行了工作流程	`workflow_dispatch`
工作流每晚在特定时间运行	`计划`（cron）
工作流在外部服务调用后运行	`存储库调度`
当问题被标记或评论时，工作流程就会运行	`问题。*` 事件

通过查看时间戳、拉取请求活动或提交历史记录，您通常可以查明导致工作流程运行的操作。

© 版权所有 Microsoft Corporation。保留所有权利。

概括

要确定触发工作流程的原因：

- 在“操作”选项卡中检查工作流程运行摘要。
- 在工作流中打印或记录github.event_name以提高可见性。
- 比较时间戳和 repo 活动（提交、PR、问题）来推断触发器。
- 使用完整的事件背景进行更深入的调查。

这些实践有助于调试、审核和提高整个开发和部署流程的工作流可靠性。

© 版权所有 Microsoft Corporation。保留所有权利。

通过读取配置文件来描述工作流的效果

要描述工作流读取其配置文件的效果，您需要分析存储在.github /workflows/ 中的“ .yml ”文件的结构和内容。此文件概述了工作流的运行时间、功能以及在不同条件下的行为方式。

解释工作流程的效果：

1

识别触发器 (on:) 此部分会告诉您工作流何时启动。例如：

```
on:
  push:
    branches: [main]
  pull_request:
    types: [opened, synchronize]
  workflow_dispatch:
```

影响：

- 当代码推送到主分支时自动运行。
- 在创建或更新拉取请求时运行。
- 也可以由用户手动触发

© 版权所有 Microsoft Corporation。保留所有权利。

通过读取工作流的配置文件来描述其效果（续）

2

了解作业和步骤（作业：）作业描述了工作流程将执行的操作。例如：

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
```

影响：

- 使用 Linux 虚拟环境（ubuntu-latest）。
- 检出存储库的代码。
- 安装项目依赖项。
- 运行自动化测试。

© 版权所有 Microsoft Corporation。保留所有权利。

通过读取工作流的配置文件来描述其效果（续）

3

评估目的和结果

- 通过阅读配置，您可以描述工作流程的预期结果：“此工作流程是一个持续集成（CI）流水线。它确保推送到存储库或通过拉取请求提交的任何新代码都经过自动测试。如果测试失败，工作流程将在 GitHub UI 中指示这一点，以帮助维护代码质量。”

4

影响行为的可选功能

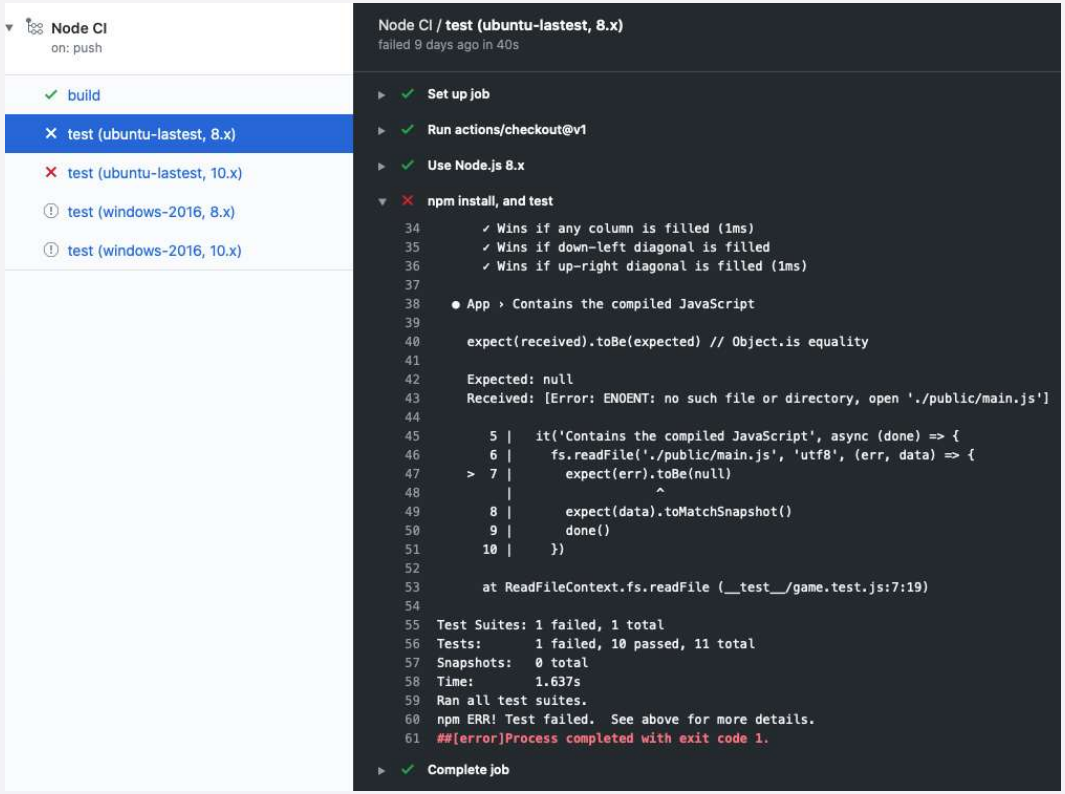
- env：设置环境变量。
- if：添加条件逻辑，仅当满足条件时才运行某些步骤。
- timeout-minutes：或 continue-on-error：影响执行行为和错误处理。

© 版权所有 Microsoft Corporation。保留所有权利。

构建的操作日志

当工作流运行时，它会生成一个日志，其中包含发生的事情以及任何错误或测试失败的详细信息。

如果出现错误或测试失败，您会在日志中看到红色复选标记（而不是绿色复选标记）。您可以检查错误或失败的详细信息，以调查发生了什么。



© 版权所有 Microsoft Corporation。保留所有权利。

诊断失败的工作流程运行

1

转到“操作”选项卡

导航到存储库的“操作”选项卡，然后：

- 找到失败的运行（通常用红色标记）
- 单击失败的工作流程以打开运行摘要。

2

查看日志中的错误

在运行摘要中：

- 展开每个作业和步骤，直到找到标记为失败的作业和步骤。
- 单击即可查看其日志。

寻找：

- 错误消息
- 堆栈跟踪
- 退出代码

例如，失败的测试可能会显示npm ERR！测试失败。或退出代码 1。

© 版权所有 Microsoft Corporation。保留所有权利。

诊断失败的工作流程运行（续）

3

检查工作流配置文件

使用 .yml文件确定：

- 每一步都试图做什么？
- 如果存在影响执行的环境变量（env:）或条件（if:）。
- 如果失败是由于缺少依赖项、语法错误或步骤配置错误造成的。

如果此步骤失败，请检查：

- 上一步依赖项是否安装成功？
- 测试文件是否存在并在本地通过？

4

常见故障场景

症状	可能的原因
步骤失败，`未找到命令`	缺少依赖项或设置错误
`npm安装` 失败	损坏的`包-lock.json`，网络问题
测试步骤失败	单元测试问题，缺少配置或无效的测试语法
`没有权限`	文件权限不正确或机密信息丢失

© 版权所有 Microsoft Corporation。保留所有权利。

确定从用户界面访问工作流日志的方法

1

前往存储库

导航到包含工作流的存储库。

2

单击“操作”选项卡

导航到位于存储库顶部导航栏的“操作”选项卡。此选项卡显示该存储库所有工作流程运行的历史记录。

3

选择工作流名称

从列表中选择相关的工作流程。例如，如果您的.yml文件包含以下内容：

```
name: CI Workflow
```

您将在列表中看到一个名为 CI Workflow 的链接。

© 版权所有 Microsoft Corporation。保留所有权利。

确定从用户界面访问工作流日志的方法

4

选择特定的运行

- 您将看到带有状态指示器的运行列表
- 单击您要检查的特定运行的时间戳或提交消息。

5

展开每个作业和步骤

- 运行摘要页面显示工作流文件中定义的作业（例如，构建、测试）。
- 单击某个作业即可将其展开。
- 在作业内部，展开各个步骤（例如，“安装依赖项”、“运行测试”）。

6

查看日志输出

- 单击某个步骤将显示完整的日志输出（例如，控制台日志、错误消息、调试信息）。
- 您可以复制、搜索或下载这些日志。

© 版权所有 Microsoft Corporation。保留所有权利。

概括

行动	目的
“操作”选项卡	查看所有工作流程运行
选择工作流名称	按工作流程过滤运行
点击运行	查看具体工作/步骤的结果
展开步骤	查看详细日志
下载日志	用于离线或团队故障排除

© 版权所有 Microsoft Corporation。保留所有权利。

自定义工作流模板

在本模块的开头，我们描述了一个需要为团队设置 CI 的场景。Node.js 模板是一个很好的起点，但您需要对其进行自定义，以更好地满足您团队的需求。您希望针对不同的 Node 版本和不同的操作系统进行开发。您还希望将构建和测试步骤作为单独的作业。

让我们看看如何自定义工作流程。

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node-version: [16.x, 18.x]
```

在这里，我们配置了一个构建矩阵 用于跨多个操作系统和语言版本进行测试。此矩阵生成四个版本，每个操作系统对应一个 Node 版本。

© 版权所有 Microsoft Corporation。保留所有权利。

自定义工作流模板（续）

四个构建及其所有测试会产生大量的日志信息。整理这些信息可能比较困难。在以下示例中，我们将向您展示如何将测试步骤移至专用测试作业。此作业针对多个目标进行测试。将构建步骤和测试步骤分开，可以更轻松地理解日志。

```yaml
test:
  runs-on: ${{ matrix.os }}
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest]
      node-version: [16.x, 18.x]
  steps:
  - uses: actions/checkout@v3
  - name: Use Node.js ${{ matrix.node-version }}
    uses: actions/setup-node@v3
    with:
      node-version: ${{ matrix.node-version }}
  - name: npm install, and test
    run: |
      npm install
      npm test
    env:
      CI: true
```

© 版权所有 Microsoft Corporation。保留所有权利。

什么是工件？

当工作流程生成除日志条目之外的内容时，该产品称为*工件*。例如，Node.js 构建会生成一个可部署的 Docker 容器。您可以使用操作actions/upload-artifact将此工件（即容器）上传到存储。 然后使用操作actions/download-artifact从存储中下载。

存储工件会在作业之间保留它。每个作业都会使用虚拟机 (VM) 的全新实例，因此您无法通过将工件保存在 VM 上来重复使用工件。如果您在其他作业中需要该工件，则可以在一个作业中将工件上传到存储，然后在另一个作业中下载它。

工件存储

工件存储在 GitHub 的存储空间中。公共仓库可免费使用存储空间，私有仓库也可免费使用一定数量的存储空间，具体取决于帐户。GitHub 会存储您的工件 90 天。

© 版权所有 Microsoft Corporation。保留所有权利。

工件存储（续）

在以下工作流代码片段中，请注意`actions/ upload-artifact@main`操作中有一个`path:`属性。此属性的值是存储工件的路径。在这里，我们指定 public/ 来将所有内容上传到目录。如果我们只想上传单个文件，则可以使用类似*public/ mytext.txt的路径*。

```
build:
   runs-on: ubuntu-latest
   steps:
     - uses: actions/checkout@v3
     - name: npm install and build webpack
       run: |
         npm install
         npm run build
     - uses: actions/upload-artifact@main
       with:
         name: webpack artifacts
         path: public/
```

© 版权所有 Microsoft Corporation。保留所有权利。

工件存储（续）

要下载用于测试的工件，构建必须成功完成并上传该工件。在下面的代码中，我们指定测试作业依赖于构建作业。

```
test:
    needs: build
    runs-on: ubuntu-latest
```

在以下工作流片段中，我们下载了该工件。现在，测试作业可以使用该工件进行测试。

```
steps:
    - uses: actions/checkout@v3
    - uses: actions/download-artifact@main
      with:
        name: webpack artifacts
        path: public
```

有关在工作流中使用工件的更多信息，请参阅将工作流数据存储为工件 在 GitHub 文档中。

© 版权所有 Microsoft Corporation。保留所有权利。

在作业之间传递工件数据

与在工作流中缓存依赖项的思路类似，您可以在同一工作流中的作业之间传递数据。您可以使用`upload-artifact`和`download-artifact`操作来实现这一点。依赖于前一个作业的工件的作业必须等待前一个作业成功完成才能运行。如果您有一系列作业需要根据从前一个作业上传的工件按顺序运行，则此功能非常有用。例如，`job_2`需要`job_1`，方法是使用`needs: job_1`语法。

上述示例有两个作业。`job_1`将一些文本写入文件`file.txt`。然后它使用`actions/upload-artifact@v2`上传此工件并存储数据以供将来在工作流中使用。job_2需要`job_1`通过使用需求来完成：`job_1`语法。然后，它使用`actions/download-artifact@v2`操作下载该工件，然后打印`file.txt`的内容。

```yaml
name: Share data between jobs
on: push
jobs:
  job_1:
    name: Upload File
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello World" > file.txt
      - uses: actions/upload-artifact@v2
        with:
          name: file
          path: file.txt

  job_2:
    name: Download File
    runs-on: ubuntu-latest
    needs: job_1
    steps:
      - uses: actions/download-artifact@v2
        with:
          name: file
      - run: cat file.txt
```

© 版权所有 Microsoft Corporation。保留所有权利。

使用工作流程在 GitHub 中自动执行审核

有时，我们只想在用户执行操作后运行工作流。例如，我们可能只想在审阅者批准拉取请求后运行工作流。对于这种情况，我们可以在“拉取请求审阅”上触发。

我们可以采取的另一个操作是为拉取请求添加标签。在本例中，我们使用pullreminders/label-when-approved-action 行动。

```yaml
steps:
     - name: Label when approved
       uses: pullreminders/label-when-approved-action@main
       env:
         APPROVALS: "1"
         GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
         ADD_LABEL: "approved"
```

注意名为 env: 的代码块。此代码块用于设置此操作的环境变量。例如，您可以设置所需的审批者数量。此处设置为 1。secrets.GITHUB_TOKEN身份验证变量是必需的，因为此操作必须通过添加标签来更改您的存储库。最后，您需要提供要添加的标签的名称。

© 版权所有 Microsoft Corporation。保留所有权利。

使用环境变量和工件数据自定义工作流程

在这里，您将学习如何使用默认和自定义环境变量、自定义脚本、缓存依赖项以及在作业之间传递工件数据。您还将学习如何从 GitHub 网站和 REST API 端点访问工作流日志。

© 版权所有 Microsoft Corporation。保留所有权利。

默认环境变量和上下文

在 GitHub Actions 工作流程中，有多个默认环境变量可供您使用，但仅限于执行作业的运行器内。这些默认变量区分大小写，它们引用系统和当前用户的配置值。我们建议您使用这些默认环境变量来引用文件系统，而不是使用硬编码的文件路径。要使用默认环境变量，请指定 $ 后跟环境变量的名称。

```
jobs:
  prod-check:
    steps:
      - run: echo "Deploying to production server on branch $GITHUB_REF"
```

除了默认环境变量之外，您还可以使用已定义的变量作为上下文。上下文和默认变量的相似之处在于它们都提供对环境信息的访问，但它们之间存在一些重要区别。默认环境变量只能在运行器中使用，而上下文变量可以在工作流程中的任何位置使用。

© 版权所有 Microsoft Corporation。保留所有权利。

默认环境变量和上下文（续）

例如，上下文变量允许您在执行运行器之前运行 if 语句来评估表达式。

```yaml
name: CI
on: push
jobs:
  prod-check:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to production server on branch $GITHUB_REF"
```

此示例使用`github.ref`上下文来检查触发工作流程的分支。如果分支是`main`，则执行运行器并打印“正在部署到分支 $GITHUB_REF 上的生产服务器”。运行器中使用默认环境变量`$GITHUB_REF`来引用分支。请注意，默认环境变量全部大写，而上下文变量全部小写。

© 版权所有 Microsoft Corporation。保留所有权利。

工作流中可用的上下文信息

语境	描述
GitHub	有关工作流程运行的信息。有关更多信息，请参阅 github 上下文。
环境	包含在工作流、作业或步骤中设置的变量。有关更多信息，请参阅env context 。
变量	包含在仓库、组织或环境级别设置的变量。有关更多信息，请参阅vars context 。
工作	有关当前正在运行的作业的信息。有关更多信息，请参阅作业上下文。
工作	仅适用于可重用工作流程，包含可重用工作流程的作业输出。有关更多信息，请参阅作业上下文。
步骤	有关当前作业中运行的步骤的信息。有关更多信息，请参阅步骤上下文。
跑步者	有关正在运行当前作业的运行器的信息。有关更多信息，请参阅运行器上下文。
秘密	包含工作流程运行可用的密钥的名称和值。有关更多信息，请参阅密钥上下文。
战略	有关当前作业的矩阵执行策略的信息。有关更多信息，请参阅策略上下文。
矩阵	包含工作流中定义的适用于当前作业的矩阵属性。有关更多信息，请参阅矩阵上下文。
需求	包含定义为当前作业依赖项的所有作业的输出。有关更多信息，请参阅需求上下文。
输入	包含可重复使用或手动触发的工作流的输入。有关更多信息，请参阅输入上下文。

© 版权所有 Microsoft Corporation。保留所有权利。

工作流程中可用的上下文信息（续）

Workflow Key	Context	Special Functions
run-name	github, inputs, vars	None
concurrency	github, inputs, vars	None
env	github, secrets, inputs, vars	None
jobs.<job_id>.concurrency	github, needs, strategy, matrix, inputs, vars	None
jobs.<job_id>.container	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.container.credentials	github, needs, strategy, matrix, env, vars, secrets, inputs	None
jobs.<job_id>.container.env.<env_id>	github, needs, strategy, matrix, job, runner, env, vars, secrets, inputs	None
jobs.<job_id>.container.image	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.continue-on-error	github, needs, strategy, vars, matrix, inputs	None
jobs.<job_id>.defaults.run	github, needs, strategy, matrix, env, vars, inputs	None
jobs.<job_id>.env	github, needs, strategy, matrix, vars, secrets, inputs	None
jobs.<job_id>.environment	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.environment.url	github, needs, strategy, matrix, job, runner, env, vars, steps, inputs	None
jobs.<job_id>.if	github, needs, vars, inputs	always, canceled, success, failure
jobs.<job_id>.name	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.outputs.<output_id>	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	None
jobs.<job_id>.runs-on	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.secrets.<secrets_id>	github, needs, strategy, matrix, secrets, inputs, vars	None
jobs.<job_id>.services	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.services.<service_id>.credentials	github, needs, strategy, matrix, env, vars, secrets, inputs	None
jobs.<job_id>.services.<service_id>.env.<env_id>	github, needs, strategy, matrix, job, runner, env, vars, secrets, inputs	None
jobs.<job_id>.steps.continue-on-error	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.env	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.if	github, needs, strategy, matrix, job, runner, env, vars, steps, inputs	always, canceled, success, failure, hashFiles
jobs.<job_id>.steps.name	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.run	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.timeout-minutes	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.with	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.steps.working-directory	github, needs, strategy, matrix, job, runner, env, vars, secrets, steps, inputs	hashFiles
jobs.<job_id>.strategy	github, needs, vars, inputs	None
jobs.<job_id>.timeout-minutes	github, needs, strategy, matrix, vars, inputs	None
jobs.<job_id>.with.<with_id>	github, needs, strategy, matrix, inputs, vars	None
on.workflow_call.inputs.<inputs_id>.default	github, inputs, vars	None
on.workflow_call.outputs.<output_id>.value	github, jobs, vars, inputs	None

© 版权所有 Microsoft Corporation。保留所有权利。

在工作流中设置自定义环境变量

在工作流文件顶层使用`env`定义作用域为整个工作流的环境变量。使用`jobs.< job_id >.env`定义工作流中作业内容的作用域。您也可以使用`jobs.< job_id >.steps[*].env`定义作业中特定步骤的环境变量的作用域。

接下来是在工作流文件中显示所有三种场景的示例:

```yaml
name: Greeting on variable day
on:
  workflow_dispatch
env:
  DAY_OF_WEEK: Monday
jobs:
  greeting_job:
    runs-on: ubuntu-latest
    env:
      Greeting: Hello
    steps:
      - name: "Say Hello Mona it's Monday"
        run: echo "$Greeting $First_Name. Today is $DAY_OF_WEEK!"
        env:
          First_Name: Mona
```

© 版权所有 Microsoft Corporation。保留所有权利。

在工作流中使用默认上下文

默认环境变量由 GitHub 设置，并非在工作流中定义。它们可在工作流的相应上下文中使用。除`CI`外，大多数默认变量都以`GITHUB_*`或`RUNNER_*`开头。后两种类型无法被覆盖。此外，这些默认变量还具有相应的、名称相似的 context 属性。例如，`RUNNER_*`系列默认变量具有匹配的 context 属性`runner.*`。您可以在此处查看应用这些方法在工作流中访问默认变量的示例：

```yaml
on: workflow_dispatch

jobs:
  if-Windows-else:
    runs-on: macos-latest
    steps:
      - name: condition 1
        if: runner.os == 'Windows'
        run: echo "The operating system on the runner is $env:RUNNER_OS."
      - name: condition 2
        if: runner.os != 'Windows'
        run: echo "The operating system on the runner is not Windows, it's $RUNNER_OS."
```

有关详细信息，请参阅默认环境变量 在 GitHub 用户文档中。

© 版权所有 Microsoft Corporation。保留所有权利。

将自定义环境变量传递给工作流

您可以将客户环境变量从工作流作业的一个步骤传递到作业中的后续步骤。在作业的一个步骤中生成一个值，并将其赋值给现有或新的环境变量。接下来，将变量/值对写入 GITHUB_ENV 环境文件。该环境文件可以由操作使用，也可以通过使用`run`关键字在工作流作业中的 shell 命令中使用。

创建或更新环境变量的步骤无权访问新值，但作业中的所有后续步骤都可以访问。

您可以在此处查看示例：

```yaml
steps:
  - name: Set the value
    id: step_one
    run: |
      echo "action_state=yellow" >> "$GITHUB_ENV"
  - name: Use the value
    id: step_two
    run: |
      printf '%s\n' "$action_state" # This will output 'yellow'
```

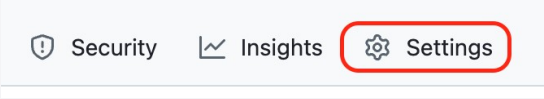
© 版权所有 Microsoft Corporation。保留所有权利。

添加环境保护

您可以为 GitHub 仓库定义的环境添加保护规则。要添加环境，请在您的仓库中执行以下操作：

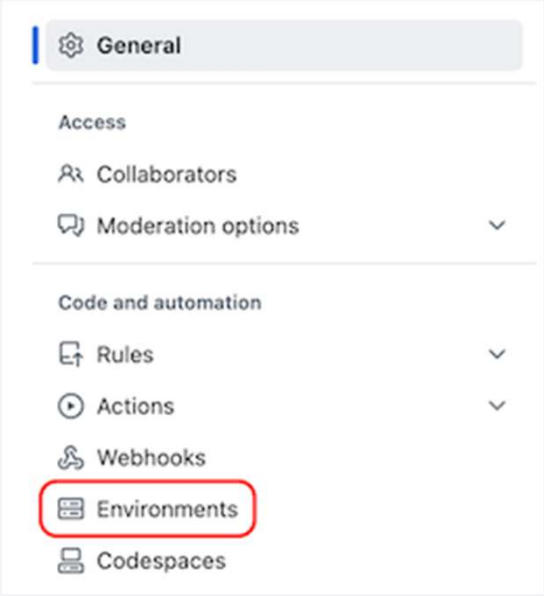
1

点击“设置”



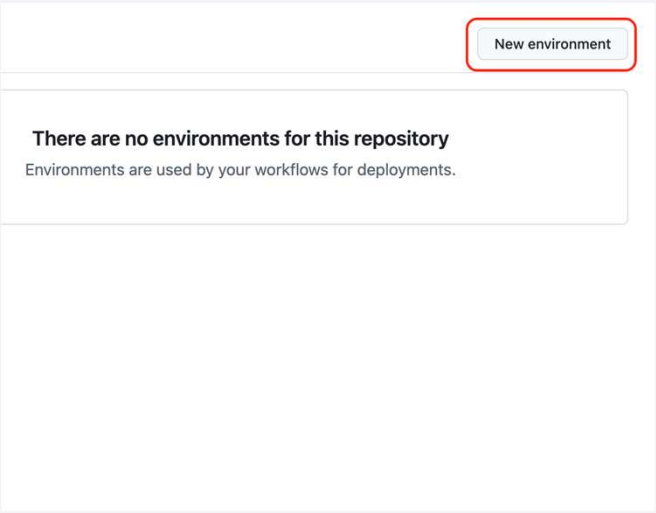
2

单击左侧面板上的“环境”。



3

使用**新环境**按钮添加和配置环境并添加保护。



© 版权所有 Microsoft Corporation。保留所有权利。

关于环境

环境用于描述一般部署目标，例如生产、暂存或开发。当 GitHub Actions 工作流程部署到某个环境时，该环境将显示在仓库的主页上。您可以使用环境来要求作业获得批准才能继续进行、限制哪些分支可以触发工作流程、使用自定义部署保护规则控制部署，或限制对机密的访问。

工作流中的每个作业都可以引用单个环境。在将引用该环境的作业发送给运行器之前，必须先通过为该环境配置的任何保护规则。只有在将作业发送给运行器后，该作业才能访问该环境的机密。

当工作流引用某个环境时，该环境就会出现在存储库的部署中。

© 版权所有 Microsoft Corporation。保留所有权利。

环境保护法规

环境部署保护规则要求在引用该环境的作业继续之前满足特定条件。您可以使用部署保护规则来要求手动批准、延迟作业或将环境限制在某些分支上。您还可以创建和实施由 GitHub 应用提供支持的自定义保护规则，以使用第三方系统来控制引用 GitHub 上配置的环境的部署。以下是这些保护规则的说明：

1

必需的审阅者保护规则：

使用必需审阅者来要求特定人员或团队批准引用该环境的工作流作业。您最多可以列出六个用户或团队作为审阅者。审阅者必须至少拥有存储库的读取权限。只需其中一位必需审阅者批准该作业即可继续进行。

您还可以选择阻止对部署到受保护环境的部署进行自我审核。如果启用此设置，启动部署的用户将无法批准部署作业，即使他们是必需的审核者。通过启用自我审核，可以确保部署到受保护环境的部署始终由多人审核。

有关审查引用具有所需审查者的环境的作业的更多信息，请参阅审查部署。

© 版权所有 Microsoft Corporation。保留所有权利。

环境保护规则（续）

2

等待计时器投影规则：

您可以使用等待计时器保护规则，在作业首次触发后，将其延迟特定时间，然后才允许继续进行环境部署。该时间（以分钟为单位）必须是 1 到 43,200（30 天）之间的整数。等待时间不计入您的计费时间。

分支和标签保护规则：

您可以使用部署分支和标签保护规则来限制用于部署到环境的分支和标签。对于一个环境，您有多种部署分支和标签保护规则选项。

3

无限制表示不限制哪些分支或标签可以部署到环境。**受保护的分支仅**允许启用了分支保护规则的分支部署到环境。如果存储库中的任何分支均未定义分支保护规则，则所有分支都可以部署。**选定的分支和标签**设置确保只有符合您指定名称模式的分支和标签才能部署到环境。

如果您指定`releases/*`作为部署分支或标签规则，则只有名称以`releases/`开头的分支或标签才能部署到环境。（通配符与`/`不匹配。要匹配以`release/`开头且包含一个附加单斜杠的分支或标签，请使用`release/*/*` 。）如果您添加`main`作为分支规则，则名为`main`的分支也可以部署到环境。

© 版权所有 Microsoft Corporation。保留所有权利。

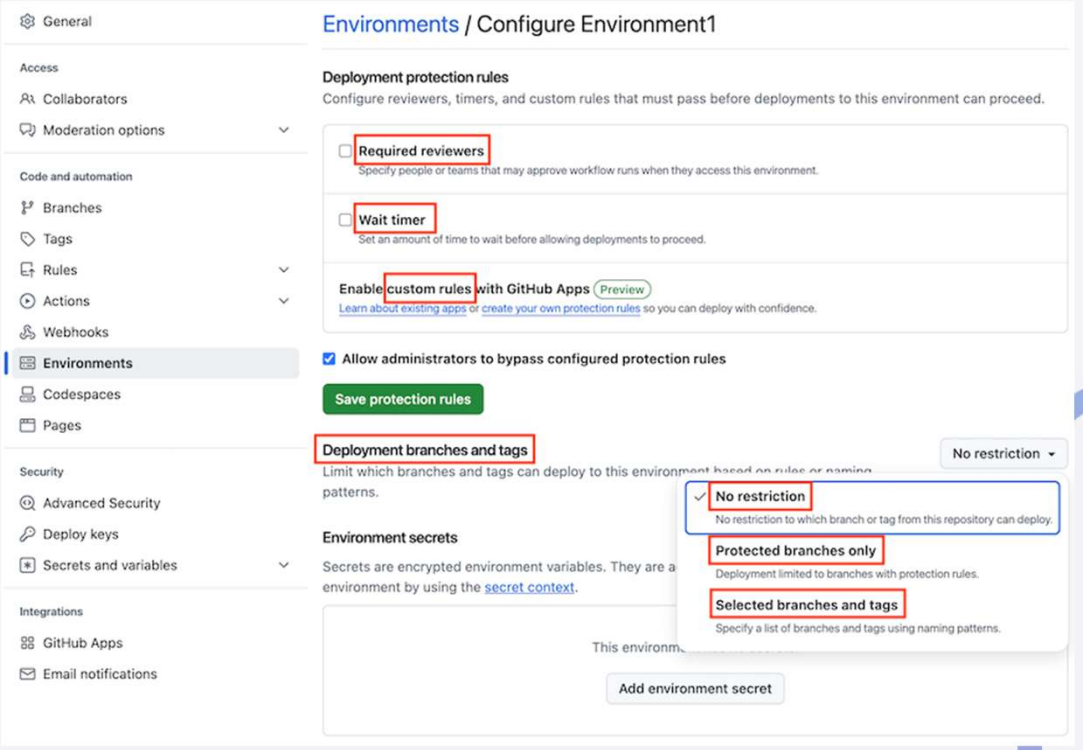
环境保护规则（续）

4

自定义部署保护规则：

您可以启用自定义保护规则，以控制使用第三方服务的部署。例如，您可以使用可观察性系统、变更管理系统、代码质量系统或其他手动配置来评估准备情况，并为部署到 GitHub 提供自动审批。

一旦在存储库上创建并安装了自定义部署保护规则，您就可以为存储库中的任何环境启用自定义部署保护规则。



注意：如果您使用的是 GitHub Free、GitHub Pro 或 GitHub Team 套餐，则环境部署投影规则仅适用于公共仓库；分支和标签保护规则除外。对于 GitHub Pro 或 GitHub Team 套餐的用户，分支和标签保护规则也适用于私有仓库。

© 版权所有 Microsoft Corporation。保留所有权利。

工作流程中的脚本

在前面的工作流代码片段示例中， run关键字用于打印一串文本。由于run关键字指示作业在运行器上执行命令，因此您可以使用 run 关键字来运行操作或脚本。

```
jobs:
  example-job:
    steps:
      - run: npm install -g bats
```

在此示例中，您将使用npm 的run关键字安装bats软件测试包。您也可以将脚本作为操作运行。您可以将脚本存储在存储库中（通常存储在.github /scripts/ 目录中），然后使用run关键字提供路径和 shell 类型。

```
jobs:
  example-job:
    steps:
      - name: Run build script
        run: ./.github/scripts/build.sh
        shell: bash
```

© 版权所有 Microsoft Corporation。保留所有权利。

缓存操作的缓存依赖项

构建工作流程时，您经常会发现需要重复使用相同的输出，或者在一次运行之间下载依赖项。与其反复下载这些依赖项，不如缓存它们，这样可以使工作流程运行得更快、更高效。缓存依赖项可以减少工作流程中某些步骤的运行时间，因为 GitHub 托管的运行器上的作业每次都会在干净的虚拟环境中启动。

要缓存作业的依赖项，请使用 GitHub 的`cache`操作。此操作会检索由您提供的唯一键标识的缓存。当该操作找到缓存后，它会将缓存的文件检索到您配置的路径。要使用`cache`操作，您需要设置一些特定参数：

范围	描述	必需的
Key	指保存和搜索缓存时创建的关键标识符。	是的
Path	指运行器上要缓存或搜索的文件路径。	是的
Secret	如果未找到所需的缓存键，则由缓存的替代现有键组成。	不

© 版权所有 Microsoft Corporation。保留所有权利。

缓存操作的缓存依赖关系（续）

在以下示例中，路径设置为~/.npm ，密钥包含运行器的操作系统和package- lock.json文件的 SHA-256 哈希值。当您使用restore-keys后备方案并拥有多个缓存时，在密钥前添加 ID（本例中为 npm -cache ）会很有用。

```yaml
steps:
  - uses: actions/checkout@v2

  - name: Cache NPM dependencies
    uses: actions/cache@v2
    with:
      path: ~/.npm
      key: ${{ runner.os }}-npm-cache-${{ hashFiles('**/package-lock.json') }}
      restore-keys: |
        ${{ runner.os }}-npm-cache-
```

© 版权所有 Microsoft Corporation。保留所有权利。

在工作流中启用步骤调试日志记录

在某些情况下，默认工作流程日志无法提供足够的详细信息来诊断特定工作流程运行、作业或步骤失败的原因。对于这些情况，您可以为两个选项启用额外的调试日志记录：运行和步骤。通过将两个需要管理员访问权限的仓库密钥设置为`true`来启用此诊断日志记录：

- 要启用运行器诊断日志记录，请将包含工作流的存储库中的`ACTIONS_RUNNER_DEBUG`密钥设置为`true`。
- 要启用步骤诊断日志记录，请将包含工作流的存储库中的`ACTIONS_STEP_DEBUG`密钥设置为`true`。

© 版权所有 Microsoft Corporation。保留所有权利。

从用户界面访问工作流日志

当您考虑成功的自动化时，您的目标是花费最少的时间来查看哪些操作已实现自动化，以便将注意力集中在相关内容上。然而，有时事情并不按计划进行，您需要回顾发生了什么。这个调试过程可能会令人沮丧。GitHub 提供了清晰的布局结构，可以快速在作业之间导航，同时保留当前调试步骤的上下文。要查看 GitHub 中工作流程运行的日志，您可以按照以下步骤操作：

1. 导航到存储库中的**“操作”选项卡。**
2. 在左侧边栏中，单击所需的工作流程。
3. 从工作流程运行列表中，选择所需的运行。
4. **“作业”**下，选择所需的作业。
5. 读取日志输出。

如果工作流程中有多个运行，您还可以在选择工作流程后选择**“状态”过滤器并将其设置为“失败”**，以仅显示该工作流程中失败的运行。

© 版权所有 Microsoft Corporation。保留所有权利。

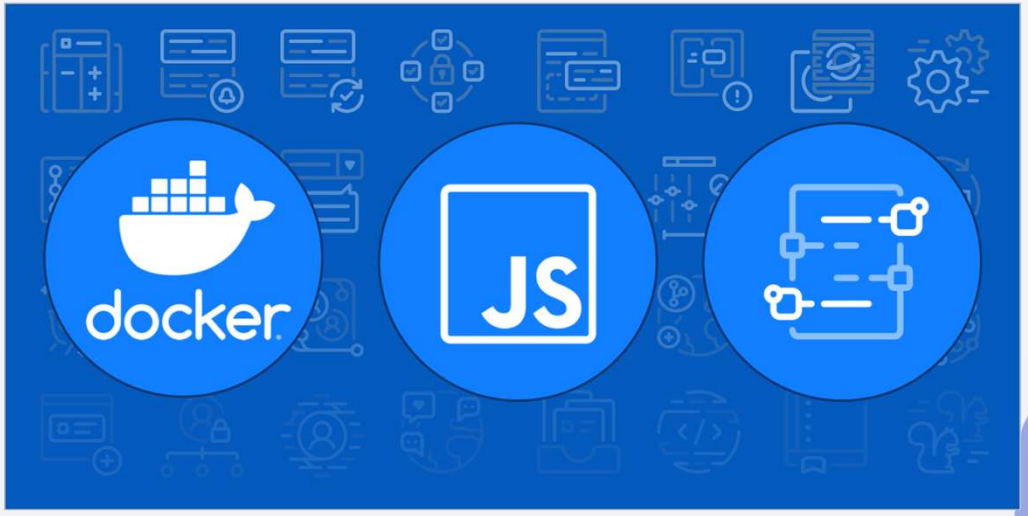
模块 3：自定义 Action 与可复用组件



© 版权所有 Microsoft Corporation。保留所有权利。

GitHub 操作的类型

操作是可用于自定义开发工作流程的各项任务。您可以编写自定义代码与仓库交互来执行自定义任务，或者使用 GitHub 社区分享的操作来创建自己的操作。



浏览各种操作，您会注意到有三种不同类型的操作：*Docker 容器操作*、*JavaScript 操作*和*复合运行步骤操作*。让我们仔细看看每种操作类型。

© 版权所有 Microsoft Corporation。保留所有权利。

Docker 容器操作

Docker 容器将环境与 GitHub Actions 代码打包在一起。这意味着操作将在一致且可靠的环境中运行，因为其所有依赖项都位于该容器中。如果操作需要在特定的环境配置中运行，Docker 容器是一个不错的选择，因为您可以自定义操作系统和工具。缺点是，由于作业必须构建和检索容器，因此 Docker 容器操作通常比 JavaScript 操作慢。

在构建 Docker 容器操作之前，您应该对如何使用环境变量和 Docker 容器文件系统有一些基本的了解。构建 Docker 容器操作的步骤非常简单明了：

1. 创建一个`Dockerfile`来定义组装 Docker 镜像的命令。
2. 创建一个`action.yml` 元数据文件来定义操作的输入和输出。在文件中将 runs:using: 的值设置为`docker` ，将 `runs:image:` 的值设置为`Dockerfile` 。
3. 创建一个`entrypoint.sh`文件来描述docker镜像。
4. 使用以下文件提交并将您的操作推送到 GitHub： `action.yml` 、 `entrypoint.sh` 、 `Dockerfile`和`README.md` 。



© 版权所有 Microsoft Corporation。保留所有权利。

JavaScript 操作

JavaScript 操作可以直接在运行器机器上运行，并将操作代码与用于运行操作的环境分离。因此，操作代码得到简化，并且执行速度比 Docker 容器内的操作更快。

作为创建和使用打包 JavaScript 操作的先决条件，您需要下载包含npm的 Node.js。可选步骤（但我们推荐）是使用 GitHub Actions Toolkit Node.js，它是 Node.js 包的集合，可让您快速构建更一致的 JavaScript 操作。

构建 JavaScript 操作的步骤非常简单且直接：

1. 创建一个`action.yml` 元数据文件来定义动作的输入和输出，以及告诉动作运行器如何开始运行这个 JavaScript 动作。
2. 创建一个`index.js` 包含有关工具包包、路由和操作的其他功能的上下文信息的文件。
3. 使用以下文件提交并将您的操作推送到 GitHub： `action.yml` 、 `index.js` 、 `node_modules` 、 `package.json` 、 `package- lock.json`和`README.md` 。



© 版权所有 Microsoft Corporation。保留所有权利。

复合运行步骤操作

复合运行步骤操作允许您使用 Shell 脚本重复使用操作。您甚至可以在同一个操作中混合使用多种 Shell 语言。如果您有多个 Shell 脚本来自动执行多项任务，现在可以轻松地将它们转换为一个操作，并在不同的工作流程中重复使用它们。有时，编写 Shell 脚本比使用 JavaScript 或将代码封装在 Docker 容器中更简单。



© 版权所有 Microsoft Corporation。保留所有权利。

打包复合动作

打包复合操作将多个步骤捆绑成一个可重复使用的单元。这些操作在存储库中定义，并可在跨不同存储库的工作流中引用。打包复合操作可以简化工作流、减少重复并提高可维护性。

创建打包的复合操作时，所有步骤都定义在一个`action.yml`文件中。该文件指定输入、输出以及要执行的命令或操作的顺序。打包的复合操作尤其适用于自动执行重复性任务或将多个 Shell 命令组合成一个可复用的操作。

© 版权所有 Microsoft Corporation。保留所有权利。

创建复合动作

1. 为复合动作设置目录

您必须将复合操作放在存储库内其自己的目录中。

示例目录结构：

```
.github/actions/my-composite-action/
├── action.yml
└── scripts/
    └── my-script.sh
```

© 版权所有 Microsoft Corporation。保留所有权利。

创建复合动作

2.定义action.yml文件

在 my-composite-action 目录中，创建一个`action.yml` 文件。**注意：** using: "composite" 字段表示此操作为复合操作。

```yaml
name: "My Composite Action"
description: "A reusable composite action that checks out code and sets up Node.js"

inputs:
  node-version:
    description: "The Node.js version to use"
    required: true

runs:
  using: "composite"
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
```

© 版权所有 Microsoft Corporation。保留所有权利。

创建复合动作

3. 在工作流中使用复合操作

一旦创建了复合操作，就可以在 GitHub Actions 工作流中引用它。

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Use my composite action
        uses: ./.github/actions/my-composite-action
        with:
          node-version: '18'
```

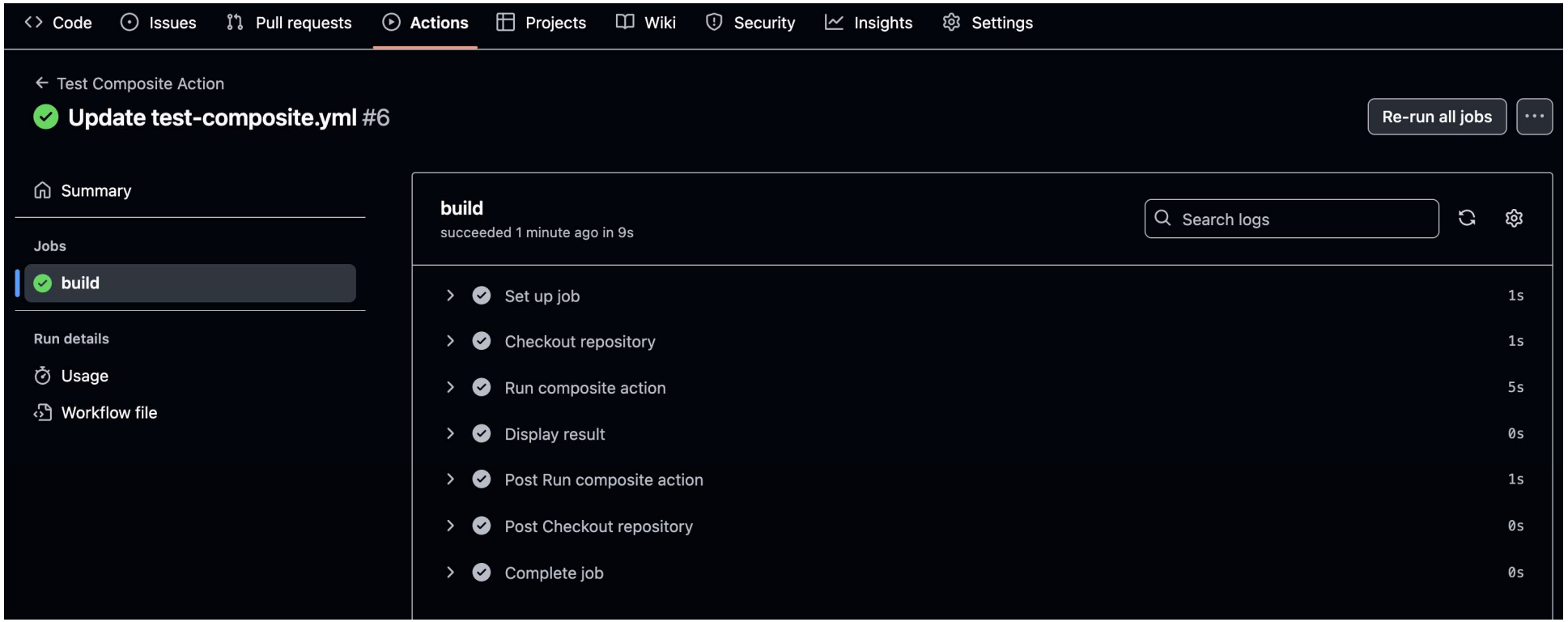
如果您的复合操作是从**另一个存储库共享的**，请像这样引用它：

```yaml
uses: owner/repository/.github/actions/my-composite-action@v1
```

© 版权所有 Microsoft Corporation。保留所有权利。

创建复合动作

查看此示例图像



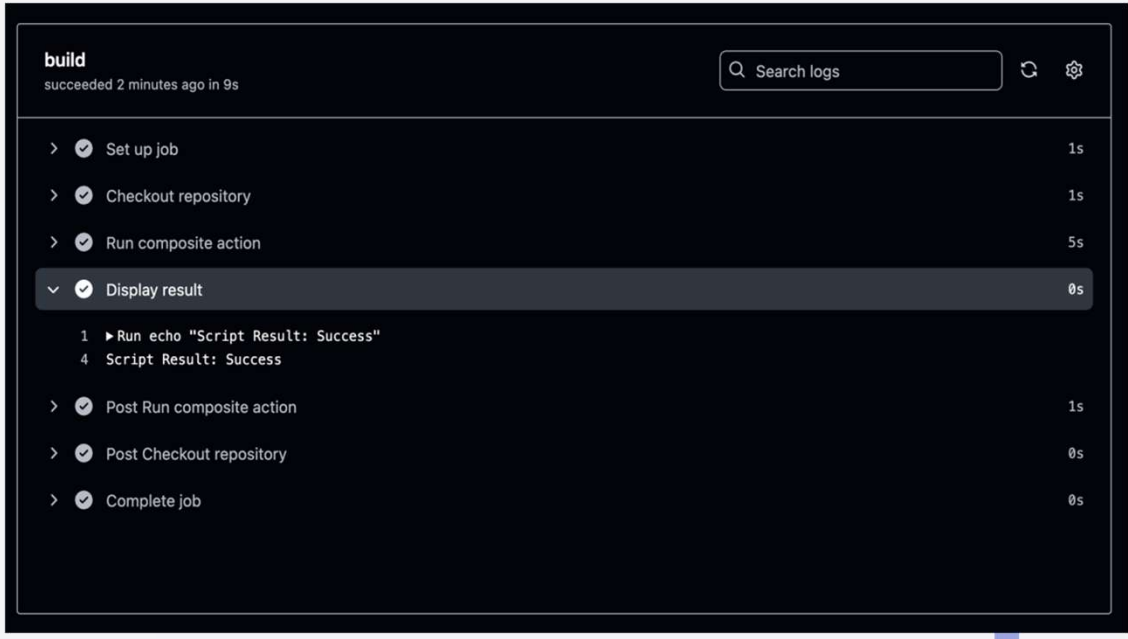
© 版权所有 Microsoft Corporation。保留所有权利。

action.yml

action.yml文件指定了一个名为script-result的输出。此输出从run-script步骤的结果输出中检索其值。run -script step 运行 Bash 命令来设置输出值。

```yaml
outputs:
  script-result:
    description: "Result from the script"
    value: ${{ steps.run-script.outputs.result }}

runs:
  using: "composite"
  steps:
    - id: run-script
      run: echo "result=Success" >> $GITHUB_OUTPUT
      shell: bash
```



© 版权所有 Microsoft Corporation。保留所有权利。

在工作流中使用输出

创建复合操作后，即可在工作流中访问其输出。以下是示例：

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run composite action
        id: my-action
        uses: ./.github/actions/my-composite-action

      - name: Display result
        run: echo "Script Result: ${{ steps.my-action.outputs.script-result }}"
```

在此示例中：

- uses关键字调用复合动作。
- 使用steps.<step-id>.outputs.<output-name>语法访问输出脚本结果。
- 在工作流日志中显示结果。

在复合操作中定义输出，以创建可重复使用的模块化工作流。这种方法简化了数据共享并提高了可维护性。

© 版权所有 Microsoft Corporation。保留所有权利。

复合操作的最佳实践

最佳实践	描述
使用版本控制	使用`v1`标签来引用稳定版本 1。
保持行动模块化	将相关步骤组合在一个复合动作中。
文档输入和输出	`action.yml`中添加输入/输出的描述。
发布前测试	在测试存储库中验证复合操作。

© 版权所有 Microsoft Corporation。保留所有权利。

工作流中的复合操作

`action.yml文件`中定义一系列命令或操作，从而更轻松地维护和跨工作流程重复使用逻辑。

复合动作的好处：

- **可重用性**——定义一次操作并在多个工作流程中使用它们。
- **可维护性**——通过将逻辑集中在单个操作中来减少重复。
- **模块化**——将多个 shell 命令或其他操作组合成一个单元。

© 版权所有 Microsoft Corporation。保留所有权利。

开发一个操作来在 GitHub Actions 运行器上设置 CLI

许多 CI/CD 工作流程需要**特定版本的 CLI 工具**来与云服务交互、管理基础架构或执行脚本。虽然 GitHub 托管的运行器预装了许多工具，但它们可能不包含您的工作流程所需的确切版本，尤其是版本较旧或不受支持的版本。您无需在每个工作流程中都安装所需的 CLI 版本，而是可以创建一个**可重复使用的 GitHub Action，**该 Action 可以：

- 确保在各个作业中一致安装所需的 CLI 版本。
- 通过集中安装逻辑简化工作流程。
- 优化缓存以加快工作流程执行速度。

© 版权所有 Microsoft Corporation。保留所有权利。

如何开发 CLI 设置操作

CLI**设置操作**是**基于 JavaScript 的操作**，用于在 GitHub 运行器上安装和配置 CLI。

在此过程中，您将通过四个简洁的步骤创建一个 CLI 设置操作。您将组织操作目录、定义其元数据、实现安装脚本，并在工作流中验证该操作。

步骤

1. 设置操作目录：在您的 repo 中创建`.github /actions/my-cli-action 。`
2. 定义`action.yml`元数据：指定名称、描述、输入和运行时（例如`node16 `）。
3. 实现安装脚本：编写`index.js`以使用Octokit或 shell 命令获取和安装 CLI 。
4. 在本地测试操作：在`.github / workflows/`下添加工作流 使用您的操作并验证 CLI 安装。`

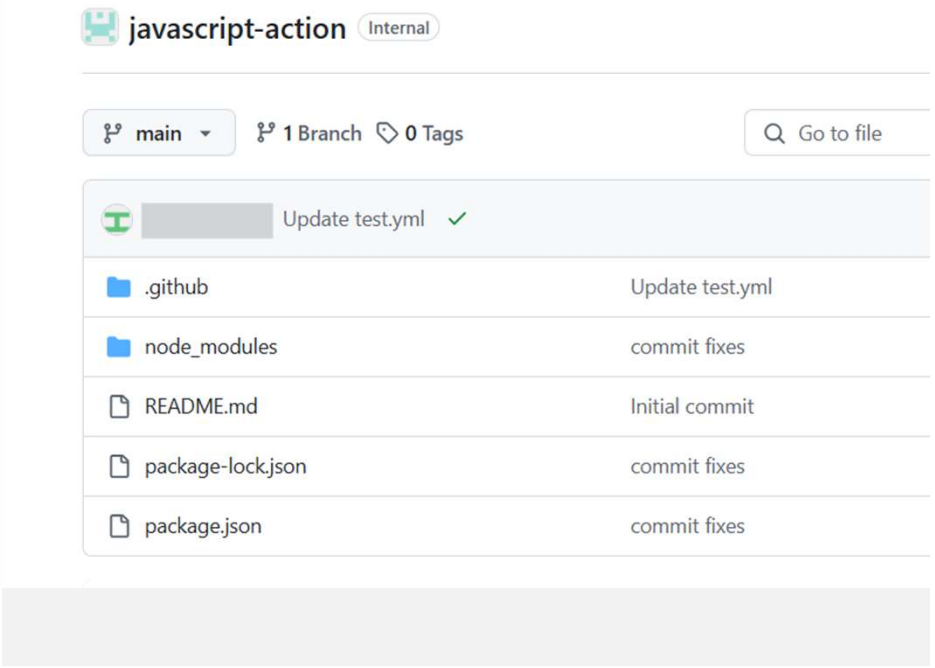
© 版权所有 Microsoft Corporation。保留所有权利。

步骤 1：设置操作目录

要为 CLI 设置操作手动创建目录，请按照以下步骤操作：

1

导航到您的存储库



2

为操作创建新目录

`.github /actions`文件夹中创建一个名为`my-cli-action`的新目录。这可以确保你的操作井然有序，并遵循 GitHub 推荐的自定义操作结构。

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 1：设置操作目录

要为 CLI 设置操作手动创建目录，请按照以下步骤操作：

3

导航到新目录

切换到新创建的目录以开始为您的操作添加文件：

4

验证目录结构

创建目录后，您的存储库结构应如下所示：

```
your-repository/
├── .github/
│   ├── actions/
│   │   ├── my-cli-action /
```

javascript-action / .github / actions / my-cli-action /

下一步：您现在可以继续创建`action.yml`文件和 CLI 设置操作所需的其他文件。

© 版权所有 Microsoft Corporation。保留所有权利。

步骤2：定义action.yml元数据文件

创建一个action.yml文件来描述该操作。

```yaml
name: "Setup MyCLI"
description: "Installs MyCLI and adds it to
the PATH"
author: "Your Name"

inputs:
  version:
    description: "The CLI version to install"
    required: false
    default: "latest"

runs:
  using: "node16"
  main: "index.js"
```

为什么要*使用 using: node16* ？ 此操作使用 Node.js 16 运行 JavaScript 代码。

javascript-action / .github / actions / my-cli-action / **action.yml**

Update action.yml

Code | Blame | 13 lines (11 loc) · 246 Bytes

```yaml
1   name: "Setup MyCLI"
2   description: "Installs MyCLI and adds it to the PATH"
3   author: "Your Name"
4
5   inputs:
6     version:
7       description: "The CLI version to install"
8       required: false
9       default: "latest"
10
11  runs:
12    using: "node16"
13    main: "index.js"
```

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 3：创建 JavaScript 脚本来安装 CLI

在同一目录下，创建一个名为index.js的文件，并添加以下代码：

JavaScript

```
const core = require('@actions/core');
const { execSync } = require('child_process');

async function run() {
  try {
    const version = core.getInput('version') || 'latest';

    console.log(`Installing MyCLI version: ${version}...`);

    execSync(`curl -fsSL https://cli.example.com/install.sh | sh`, { stdio: 'inherit' });

    console.log("MyCLI installed successfully.");
  } catch (error) {
    core.setFailed(`Installation failed: ${error.message}`);
  }
}

run();
```

JavaScript 代码使用core.getInput () 检索指定为输入的 CLI 版本。然后，它执行 curl 命令来下载并安装 CLI。如果安装过程失败，该操作将使用core.setFailed () 将工作流标记为失败。

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 3：创建 JavaScript 脚本来安装 CLI

例子：

javascript-action / .github / actions / my-cli-action / **index.js**

Create index.js

Code | Blame | 18 lines (13 loc) · 478 Bytes

```
1   const core = require('@actions/core');
2   const { execSync } = require('child_process');
3
4   async function run() {
5     try {
6       const version = core.getInput('version') || 'latest';
7
8       console.log(`Installing MyCLI version: ${version}...`);
9
10      execSync(`curl -fsSL https://cli.example.com/install.sh | sh`, { stdio: 'inherit' });
11
12      console.log("MyCLI installed successfully.");
13    } catch (error) {
14      core.setFailed(`Installation failed: ${error.message}`);
15    }
16  }
17
18  run();
```

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 4：在本地测试操作

在工作流程中使用该操作之前，请在 GitHub 托管的运行器上对其进行测试。
创建一个工作流文件（.github /workflows/ test.yml ）：

```yaml
name: Test MyCLI Setup

on:
  push:
    branches:
      - main
      - feature/*
```

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 4：在本地测试操作

1

触发工作流程

该工作流会在推送到主分支以及任何符合 feature/* 模式的分支时触发。您可以根据仓库的分支策略进行调整。

YAML

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

2

克隆存储库

actions */checkout@v4*操作用于将仓库克隆到运行器上。这确保工作流程可以访问仓库的文件。

YAML

```yaml
      - name: Install MyCLI
        uses: ./.github/actions/my-cli-action
        with:
          version: '1.2.3'
```

© 版权所有 Microsoft Corporation。保留所有权利。

步骤 4：在本地测试操作

3

运行自定义操作

uses: *./. github /actions/my-cli-action*这行在本地引用了自定义操作。请确保操作目录和action.yml文件已正确设置。version 输入指定要安装的 CLI 版本，在本例中为 1.2.3 版本。

YAML

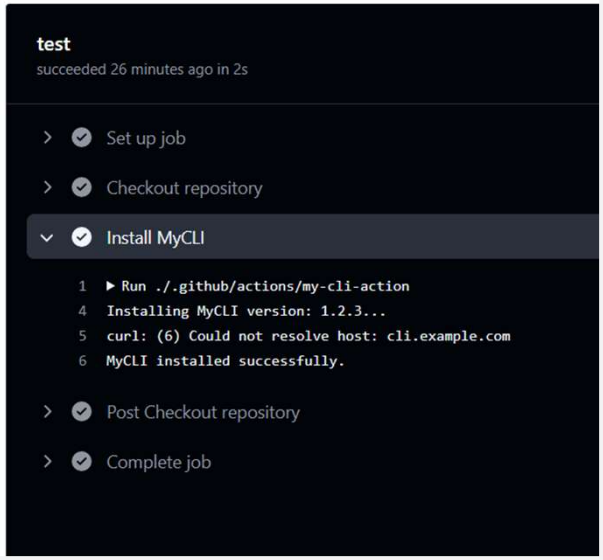
```yaml
    - name: Verify CLI Installation
      run: |
        echo "Checking MyCLI version..."
        mycli --version
```

4

验证 CLI 安装

此步骤运行 shell 命令来验证 CLI 是否已成功安装。

mycli --version 来检查已安装的 CLI 的版本。



© 版权所有 Microsoft Corporation。保留所有权利。

本地测试

要在本地测试此工作流程，请使用act CLI 工具：

```
act -j test
```

这会在您的本地机器上模拟 GitHub Actions 环境，允许您在推送更改之前调试和验证工作流程。

优化技巧：缓存

`actions/cache`操作缓存 CLI 安装目录：

```
- name: Cache MyCLI
      uses: actions/cache@v4
      with:
        path: ~/.mycli
        key: mycli-${{ runner.os }}-${{ inputs.version }}
```

这可确保后续运行重用缓存的 CLI 安装，从而减少设置时间。

© 版权所有 Microsoft Corporation。保留所有权利。

CLI 设置操作的最佳实践

最佳实践	描述
使用版本控制	`inputs.version`指定CLI版本。
正确处理错误	使用`core.setFailed ()` 出现错误时退出。
缓存 CLI 安装	`actions/cache`优化工作流程性能。
提供文件	`README.md`中解释用法和输入。

© 版权所有 Microsoft Corporation。保留所有权利。

JavaScript 操作疑难解答

使用基于 JavaScript 的 GitHub Actions 时，您可能会在工作流程执行过程中遇到意外行为、错误或失败。本单元提供一些技巧和工具，帮助您识别和解决 JavaScript 操作中的问题。

常见故障排除场景

问题	可能的原因	建议的修复
操作失败并出现堆栈跟踪	`index.js`	使用`console.log ()`并检查日志
输入`未定义`	输入名称不正确或输入缺失	验证`action.yml`以及输入如何传递
环境变量未设置	`core.exportVariable`或`process.env`未正确使用	查看设置变量的代码
未找到文件	缺少相对路径	使用`__ dirname`或文件的完整路径
缓存未恢复	错误的`键`或`路径`值	检查缓存配置和密钥

© 版权所有 Microsoft Corporation。保留所有权利。

使用日志进行调试

使用`core.info` 、 `core.debug`和 `控制台日志`

JavaScript

```
const core = require('@actions/core');

core.info("This is an info message");
core.debug("This is a debug message");
console.log("This is a raw console log");
```

✅ 使用core.debug进行调试日志 – – 仅当 ACTIONS_STEP_DEBUG 设置为 true 时才会出现。

启用调试日志记录

您可以通过设置以下**秘密来启用步骤级调试日志**：

```
ACTIONS_STEP_DEBUG=true
```

要启用**运行器诊断日志**，请设置：

```
ACTIONS_RUNNER_DEBUG=true
```

© 版权所有 Microsoft Corporation。保留所有权利。

设置调试机密

1. 转到您的 GitHub 存储库。
2. 导航到**“设置”>“机密和变量”>“操作”**。
3. 添加具有以下名称和值的新机密：
   - **操作步骤调试：** 真的
   - **ACTIONS_RUNNER_DEBUG**：真

检查工作流日志

当工作流失败时，请在“操作”选项卡中单击失败的作业。展开每个步骤以执行以下操作：

- 查看详细日志
- 检查标准输出（stdout）
- 查看脚本的退出代码
- 识别未处理的异常

🔍 示例日志输出

```plaintext
Error: Cannot find module '@actions/core'
Require stack:
- /home/runner/work/_actions/my-org/my-action/index.js
```

✅ 修复：运行npm install @actions/core 并提交node_modules （或使用ncc捆绑操作）。

© 版权所有 Microsoft Corporation。保留所有权利。

在本地测试您的操作

使用 act – 一个 CLI 工具，在本地运行 GitHub Actions。示例：

```sh
act -j test
```

🛠️ 确保在本地测试时正确模拟您的 GitHub 环境。

优雅地处理错误

捕获异常并显示有用的失败消息：

```JavaScript
try {
  // your logic
} catch (error) {
  core.setFailed(`Action failed with error: ${error.message}`);
}
```

🔁 这可确保 GitHub 在发生错误时停止工作流程并提供可读的日志。

© 版权所有 Microsoft Corporation。保留所有权利。

调试 JavaScript 操作的最佳实践

实践	描述
使用 core.debug()	除非启用调试，否则隐藏详细日志。
验证 action.yml	确保输入和输出定义正确。
捆绑代码	使用`@ vercel / ncc`将 JavaScript 编译为单个文件。这减少了依赖关系并确保包含所有必需的模块，从而防止因文件缺失而导致的运行时错误。
用行动来测试	在本地模拟运行以实现更快的迭代。
使用 try/catch	防止工作流程悄然失败。

© 版权所有 Microsoft Corporation。保留所有权利。

排查 Docker 容器操作问题

Docker 容器操作功能强大，可用于封装 GitHub Actions 工作流程中的复杂工具和环境。然而，由于 Docker 容器操作的运行时环境相互隔离，因此调试这些操作可能比 JavaScript 操作更具挑战性。

Docker 容器操作中的常见问题

问题	可能的原因	建议的修复
操作启动失败	`ENTRYPOINT`或`CMD`配置错误	确认Dockerfile正确使用了`ENTRYPOINT`
缺少依赖项	依赖项未安装或配置错误	确保所有软件包都安装在镜像中
未收到输入	`输入_` 未访问环境变量	使用`process.env.INPUT_<INPUT_NAME>`（或等效的 shell）
未找到文件	容器内的文件路径不正确	使用绝对路径或验证目录结构
没有权限	文件或脚本缺少执行权限	在Dockerfile中
网络相关故障	外部服务无法访问	验证网络设置和重试逻辑

© 版权所有 Microsoft Corporation。保留所有权利。

了解 Docker 操作生命周期

在进行故障排除之前，了解 Docker 容器操作的运行方式会很有帮助。

1

工作流触发器

GitHub Actions 工作流程响应配置的事件（例如`push`、`pull_request`或手动`workflow_dispatch`）而启动。

2

跑步者设置

GitHub 预置了一个新的虚拟机（运行器）来执行工作流。运行器通过下载操作定义和解析依赖关系来准备环境。

3

行动决议

如果操作指定了`runs.using: docker`在其`action.yml`中文件，GitHub 将其识别为基于 Docker 的操作。

© 版权所有 Microsoft Corporation。保留所有权利。

了解 Docker 操作生命周期

在进行故障排除之前，了解 Docker 容器操作的运行方式会很有帮助。

4

镜像构建或拉取

GitHub 会构建操作的 `Dockerfile中定义的 Docker` 镜像，或者拉取预构建的镜像（如果指定）。此镜像定义了操作代码的运行环境。

5

容器执行

运行器启动 Docker 容器，挂载工作区，并注入环境变量，包括工作流中定义的机密和输入。

6

入口点运行

GitHub 在容器内执行 Dockerfile中的入口点命令。自定义操作逻辑（通常是脚本或应用程序）在此运行。

© 版权所有 Microsoft Corporation。保留所有权利。

了解 Docker 操作生命周期

在进行故障排除之前，了解 Docker 容器操作的运行方式会很有帮助。

7

结果处理

容器操作设置的任何输出都会被运行器捕获，并传递给工作流中的后续步骤。完成后，容器将关闭，运行器将被丢弃。

注意： Docker 容器操作在干净、隔离的环境中运行。文件系统状态、已安装的工具和环境变量都必须在 Dockerfile中定义。

© 版权所有 Microsoft Corporation。保留所有权利。

调试技巧

1

添加日志

入口点脚本中使用 echo、printf或日志语句：

Bash

```
echo "Starting Docker action..."
echo "Input VALUE: $INPUT_VALUE"
```

记录所有关键输入和步骤以诊断故障发生的位置。

2

本地构建和测试

在您的机器上使用 Docker 模拟容器行为：

Bash

```
docker build -t my-action .
docker run -e INPUT_NAME=value my-action
```

确保环境变量模仿 GitHub 设置的变量。

3

使用 act CLI 模拟 GitHub 工作流程

安装 act 以在本地运行您的 GitHub 工作流程：

Bash

```
act -j test-job
```

非常适合在工作流中测试 Docker 操作，而无需推送到 GitHub。

© 版权所有 Microsoft Corporation。保留所有权利。

调试技巧

4

验证Dockerfile配置

确保定义了 ENTRYPOINT 或 CMD。将脚本复制到镜像中并赋予其执行权限：

Dockerfile

```
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

5

检查 GitHub 日志

单击失败的工作流程运行，展开每个步骤并检查：

- Docker 构建日志
- 容器的标准输出和标准错误
- 退出代码和堆栈跟踪 🔍 日志通常会显示缺少的包、语法问题或权限错误。

示例：入口点错误

```
Error: container_linux.go:380: starting container process caused "exec: \"/entrypoint.sh\": permission denied"
```

✅ 修复：在Dockerfile中添加 RUN chmod +x / entrypoint.sh 。

© 版权所有 Microsoft Corporation。保留所有权利。

环境变量映射

GitHub 输入	Docker 环境变量
与：名称：测试	INPUT_NAME=测试
GITHUB_WORKSPACE	映射到/ github /workspace
GITHUB_EVENT_NAME	触发工作流的事件

Bash

```
echo "Input was $INPUT_NAME"
echo "Working dir: $GITHUB_WORKSPACE"
```

© 版权所有 Microsoft Corporation。保留所有权利。

使用故障排除秘诀

通过将这些机密添加到您的存储库或组织来启用额外的日志记录：

秘密	描述
操作步骤调试	启用调试日志记录
动作运行器调试	启用跑步者诊断

© 版权所有 Microsoft Corporation。保留所有权利。

Docker 动作调试的最佳实践

最佳实践	为什么
慷慨地使用日志记录	帮助追踪故障点
始终设置 chmod +x	避免 Shell 脚本中的权限错误
验证脚本中的输入	尽早发现缺失或格式错误的输入
最小化容器依赖	较小的图像更容易调试
使用 docker run 或 act 在本地进行测试	更快的迭代和即时反馈

© 版权所有 Microsoft Corporation。保留所有权利。

发布自定义 GitHub 操作

在这里，您将了解如何为您的操作选择正确的可见性、记录和版本控制您的操作的最佳实践，以及如何将您的操作发布到 GitHub Marketplace。

© 版权所有 Microsoft Corporation。保留所有权利。

选择行动地点

创建操作时，首先要确定操作的发布位置及其可见性（无论是公开还是私有）。操作的可见性决定了发布该操作所需的建议和要求。



让我们仔细看看这两个可见性选项。

© 版权所有 Microsoft Corporation。保留所有权利。

公开



任何仓库中的工作流程都可以使用公共操作。如果您开发某个操作的目的是将其开源或通过 GitHub Marketplace 公开发布，我们建议（大多数情况下是强制要求）为该操作创建自己的仓库，而不是将其与其他应用程序代码捆绑在一起。这样，您就可以像管理其他软件一样对操作进行版本控制、跟踪和发布。这使得 GitHub 社区更容易发现该操作，缩小开发人员修复问题和扩展操作的代码库范围，并将该操作的版本控制与其他应用程序代码的版本控制区分开来。

© 版权所有 Microsoft Corporation。保留所有权利。

私人的

当操作位于私有仓库中时，只有同一仓库中的工作流程才能使用该操作。使用私有操作时，您可以将操作的文件存储在仓库中的任何位置。如果您计划将操作、工作流程和应用程序代码合并到一个仓库中，我们建议将操作存储在`.github`中 目录。例如，`.github / actions/action-a`和`.github /actions/action-` b。



© 版权所有 Microsoft Corporation。保留所有权利。

记录你的行动

如果使用新的工具或应用程序时，文档模糊甚至缺失，可能会让人非常沮丧。务必在您的操作中包含良好的文档，以便其他人能够了解其工作原理，无论您计划将其公开还是私有化。首先要做的就是创建一个良好的`README.md 文件` 为您的行动提交文件。

README.md 文件通常是开发人员查看操作工作原理的第一个地方。这里非常适合包含操作的所有重要信息。以下是需要包含的内容（非详尽列表）：

- 对操作内容的详细描述。
- 必需的输入和输出参数。
- 可选的输入和输出参数。
- 行动所使用的秘密。
- 操作使用的环境变量。
- 如何在工作流中使用您的操作的示例。

一般来说， `README.md`文件应该包含用户使用该操作所需了解的所有内容。如果您认为这些信息可能有用，请将其添加到`README.md中`。

© 版权所有 Microsoft Corporation。保留所有权利。

发布并版本化您的操作

如果您正在开发一个供他人使用的操作，无论它是公开的还是私有的，您都应该定义一个发布管理策略来控制更新的分发方式。主要版本更新，包括必要的关键修复和影响兼容性的安全补丁，都需要清晰地记录下来。

发布和版本管理的良好实践

如果您正在开发一个供他人使用的操作，无论它是公开的还是私有的，您都应该定义一个发布管理策略来控制更新的分发方式。主要版本更新，包括必要的关键修复和影响兼容性的安全补丁，都需要清晰地记录下来。

© 版权所有 Microsoft Corporation。保留所有权利。

标签

标签是管理操作发布的有效方式。通过使用标签，用户可以轻松区分主要版本和次要版本。以下列出了创建发布版本时可以考虑的一些实用做法：

- 在创建发布标签（例如 v1.0.2）之前，在发布分支（例如 release/v1）上创建并验证发布。
- 使用语义版本控制。
- 移动主要版本标签（例如 v1、v2）以指向当前版本的 Git ref。
- 为可能破坏现有工作流程的更改引入新的主要版本标签（v2）。
- 发布主要版本时，请使用 beta 标签来指示其状态；例如，v2-beta。发布准备就绪后，您可以删除 -beta 标签。

以下是每个选项的一些示例。

yml

```yml
steps:
    - uses: actions/javascript-action@v1
    - uses: actions/javascript-action@v1.0.1
    - uses: actions/javascript-action@v1-beta
```

© 版权所有 Microsoft Corporation。保留所有权利。

使用提交的 SHA

标签非常有用且应用广泛，但使用标签的一个缺点是它们可能会被删除或移动。在 Git 中，每个提交都会收到一个计算出的 SHA 值，该值是唯一的且无法修改。使用提交 SHA 进行版本控制将为您提供最可靠、最安全的版本控制和操作方法。然而，在 Git 中，您通常可以将 SHA 哈希缩写为前几个字符，Git 也会识别该引用。如果您使用提交的 SHA 进行发布管理，则需要使用完整的 SHA 值，而不是缩写值。

yml

```yml
steps:
    - uses: actions/javascript-action@2522385f6f7ba04fe7327647b213799853a8f55c
```

© 版权所有 Microsoft Corporation。保留所有权利。

将操作发布到 GitHub Marketplace



© 版权所有 Microsoft Corporation。保留所有权利。

模块 4：企业级管理与自托管运行器



© 版权所有 Microsoft Corporation。保留所有权利。

Enterprise 层级

配置 GitHub Actions 使用策略

GitHub Actions 工作流程通常包含操作，即在工作流程中执行的一组独立命令。创建工作流程时，您可以创建自己的操作来使用，也可以引用 GitHub Marketplace 提供的公共社区操作。因此，为企业中的工作流程和操作配置使用策略至关重要，以防止用户使用恶意的第三方操作。

如果您的企业设置中启用了 GitHub Connect，您可以在企业云中和企业服务器中使用多种选项来配置策略。

© 版权所有 Microsoft Corporation。保留所有权利。

Organization 层级

记录企业标准

创建 GitHub Actions 工作流程通常需要编写多个文件并创建多个仓库来指定工作流程本身。创建过程还包括创建工作流程中要使用的操作、容器和/或运行器。如果您没有制定创建 GitHub Actions 工作流程的企业标准，那么根据企业云或企业服务器实例中的用户数量，情况很快就会变得混乱。

作为最佳实践，我们建议您将以下内容记录在 GitHub wiki 中，或作为组织内所有人都可以访问的存储库中的 markdown 文件记录：

- 存储库
- 文件/文件夹命名约定
- 共享组件的位置
- 持续维护计划
- 贡献指南

© 版权所有 Microsoft Corporation。保留所有权利。

创建工作流模板

拥有组织.github仓库写入权限的用户可以创建工作流模板，以确保自动化流程在整个企业范围内得到重复使用和维护。在 Enterprise Cloud 和 Enterprise Server 中，这些用户均可将模板提供给具有相同访问权限的其他组织成员。之后，成员可以使用这些工作流模板在组织的公共和私有仓库中创建新的工作流。

创建工作流模板分两个步骤完成：

1. 创建一个yml 工作流文件。
2. 创建一个json 元数据文件，描述在用户创建工作流时应如何向用户呈现模板。
3. 您必须将这两个文件放在公共.github仓库中名为 workflow-templates 的目录中。如果您的组织中尚不存在这些目录，则可能需要创建它们。

© 版权所有 Microsoft Corporation。保留所有权利。

工作流程示例

示例yaml工作流文件：

```yaml
name: Octo Organization CI

on:
  push:
    branches: [ $default-branch ]
  pull_request:
    branches: [ $default-branch ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v2

      - name: Run a one-line script
        run: echo Hello from Octo Organization
```

笔记：
前面的文件使用`$default-branch` 占位符。当使用模板创建工作流时，此占位符将自动替换为存储库默认分支的名称。

元数据文件示例：

```
{
    "name": "Octo Organization Workflow",
    "description": "Octo Organization CI workflow
template.",
    "iconName": "example-icon",
    "categories": [
        "Go"
    ],
    "filePatterns": [
        "package.json$",
        "^Dockerfile",
        ".*\\.md$"
    ]
}
```

© 版权所有 Microsoft Corporation。保留所有权利。

控制对企业内部操作的访问

GitHub Actions 中的访问控制决定：

- **谁可以运行使用操作的工作流。**
- **哪些 Actions 可以在组织内使用。**
- **自托管运行器如何安全地执行工作流程。**
- **谁可以修改和更新共享的 GitHub Actions。**

企业管理员需要在给予开发人员灵活性和确保自动化工作流程的安全性和治理之间**取得平衡。**

组织级 Actions 使用策略

GitHub Enterprise 允许管理员设置**组织范围的策略**，以控制组织内所有仓库的操作方式。这些策略可帮助组织**限制第三方操作的使用、强制执行安全措施并标准化自动化工作流程。**

操作的存储库级别权限

虽然组织范围的设置在全球范围内适用，但**存储库级别的权限可以对谁可以运行工作流和修改操作**进行细粒度的控制。

管理存储库级别工作流权限

每个存储库可以定义**谁可以创建、编辑和执行工作流。**

权限	功能
读	查看工作流程，但无法触发或编辑它们。
写	编辑工作流程，但不能创建新的工作流程。
执行	创建、编辑和管理工作流权限。

默认情况下，GitHub 将工作流权限设置为**“读写”**，但企业应将**其限制为“读取”，除非明确要求**防止未经授权的修改。

© 版权所有 Microsoft Corporation。保留所有权利。

管理 Runner

管理企业的运行器涉及配置和保护 GitHub 托管和自托管的运行器，以确保高效安全的 CI/CD 工作流程。这包括设置 IP 允许列表来控制访问，通过限制运行器对特定 IP 地址的访问来增强安全性，以及确保符合组织政策。正确配置 GitHub 托管和自托管运行器的 IP 允许列表对于维护内部应用程序和 GitHub Actions 运行器之间的安全可靠交互至关重要。定期更新和审查这些配置对于适应 IP 地址范围的变化并保持最佳安全性至关重要。

© 版权所有 Microsoft Corporation。保留所有权利。

配置自托管运行器以供企业使用

自定义环境、网络访问和安全强化的企业提供了更大的灵活性和控制力。本指南涵盖了配置自托管运行器的最佳实践，包括**代理、标签和网络**注意事项。

1. 设置自托管运行器

步骤 1：创建并注册自托管运行器

1. 导航至**GitHub Enterprise → 设置 → 操作 → 运行器。**
2. 单击“**New Runner**”并选择所需的操作系统（**Linux、Windows 或 macOS**）。
3. 按照提供的命令在您的机器上安装和配置运行器。

步骤 2：安装并启动 Runner

根据您的操作系统运行以下命令：

Linux/macOS

```
./config.sh --url https://github.com/<org-name> --
token <generated-token>
./run.sh
```

Windows（PowerShell）

```
.\config.cmd --url https://github.com/<org-name> --
token <generated-token>
.\run.cmd
```

© 版权所有 Microsoft Corporation。保留所有权利。

配置自托管运行器以供企业使用（续）

2. 为自托管运行器配置代理

企业通常在限制互联网访问的企业防火墙和代理后面运营。要允许自托管运行器与 GitHub 通信，请按如下方式配置代理设置：

Linux/macOS：配置代理

编辑环境文件以定义代理设置：

```
export http_proxy=http://proxy.company.com:8080
export https_proxy=http://proxy.company.com:8080
export no_proxy=localhost,127.0.0.1
```

```
source ~/.bashrc
```

Windows：配置代理

使用以下 PowerShell 命令：

```
[System.Environment]::SetEnvironmentVariable("HTTP_PROXY", "http://proxy.company.com:8080", "Machine")
[System.Environment]::SetEnvironmentVariable("HTTPS_PROXY", "http://proxy.company.com:8080", "Machine")
```

© 版权所有 Microsoft Corporation。保留所有权利。

企业运营者的安全最佳实践

- **将运行器限制在受信任的工作流程中**：防止不受信任的代码在自托管运行器上执行。
- **使用临时运行器**：作业结束后自动删除运行器以防止持续威胁。
- **监控Runner活动**：记录所有Runner的动作并审核访问。
- **应用操作系统安全补丁**：定期更新并保护运行器机器。

© 版权所有 Microsoft Corporation。保留所有权利。

托管 Runner 与自托管 Runner 选型

模块 5 · 企业治理与 Runner

- **GitHub 托管**：开箱即用、自动更新、按分钟计费；受限于固定规格、公网出口与可用镜像。
- **自托管**：可访问内网资源、自定义硬件 / GPU、控制成本；但需自行负责补丁、扩容与安全。
- **决策维度**：网络可达性、合规与数据驻留、构建时长与并发量、运维能力、总体成本。
- **混合策略**：公共构建用托管 Runner，需要连内网 / 专有工具的作业路由到自托管 Runner Group。
- **标签路由**：runs-on 使用 labels 精确选择 runner 组或自托管标签。
- **安全红线**：公开仓库不得启用自托管 runner，恶意 PR 可在你的内网机器上执行代码。

© 版权所有 Microsoft Corporation。保留所有权利。

Self-Hosted Runner 三种部署模式

模块 5 · 企业治理与 Runner

- **Codespaces / 临时单机**：最快上手，适合教学与验证；进程常驻、不具备生产弹性。
- **Docker 临时 Runner**：用 --once / ephemeral 让每个作业跑在全新容器，作业结束即销毁，减少状态污染。
- **ARC Scale Set**：Actions Runner Controller 在 K8s 上按作业量自动伸缩，适合企业级弹性与资源隔离。
- **Runner Group**：按团队 / 环境分组并限制可访问的仓库，是组织级隔离的关键手段。
- **可观测性**：监控排队时长、作业成功率、runner 在线率与镜像版本一致性。
- **升级与轮换**：定期更新 runner 版本与基础镜像，避免长期运行实例积累漂移与漏洞。

© 版权所有 Microsoft Corporation。保留所有权利。

Runner 安全加固要点

模块 5 · 企业治理与 Runner

- **仓库边界**：自托管 runner 只绑定到私有仓库与指定 runner group，公开仓库一律禁用。
- **最小权限**：runner 进程使用非 root 帐户，限制容器能力与挂载，禁止访问宿主机敏感路径。
- **凭据隔离**：不在 runner 上长期存放云凭据；优先用 OIDC 短时令牌换取临时访问凭据。
- **网络出口**：限制出站白名单，记录并审计对包仓库与外部 API 的访问。
- **日志与审计**：集中收集 runner 日志、作业元数据与镜像版本，纳入 SIEM 关联分析。
- **生命周期**：临时化 + 自动轮换，作业结束销毁环境，减少持久化攻击面。

© 版权所有 Microsoft Corporation。保留所有权利。

组织级策略与 OIDC 免密

模块 5 · 企业治理与 Runner

- **Actions 使用策略**：Enterprise / Organization 层级控制是否启用 Actions、可用哪些 Action、能否创建自托管 runner。
- **工作流权限默认值**：把 GITHUB_TOKEN 默认权限统一设为只读，需要写的作业显式提权。
- **fork PR 策略**：控制来自 fork 的 PR 是否运行工作流、是否授予写权限与 Secrets。
- **OIDC 原理**：工作流用 id-token: write 申请 GitHub 签发的 OIDC token，云厂商按信任条件换取短期凭据。
- **信任条件**：按仓库、分支、环境、受众（aud）限定信任范围，防止其他仓库冒用。
- **收益**：消除长期密钥轮换负担，凭据与作业同生命周期，审计可追溯到具体运行。

© 版权所有 Microsoft Corporation。保留所有权利。

模块 5 安全集成扩展：CodeQL



© 版权所有 Microsoft Corporation。保留所有权利。

CodeQL 与供应链安全

安全扩展 · CodeQL & Supply Chain

- **code scanning 代码扫描**：基于 CodeQL 的语义分析，覆盖 JavaScript / Python / Java / Go / C# 等多语言
- **CodeQL 高级配置**：自定义查询套件 queries、路径过滤 paths、定时与 PR 触发策略
- **Secret Scanning 密钥扫描**：检测硬编码凭据；Push Protection 在推送时实时拦截泄漏
- **与 GitHub Actions 集成**：在流水线中编排 code scanning / secret scanning 作业，作为合并门禁卡点

© 版权所有 Microsoft Corporation。保留所有权利。

CodeQL 工作流示例

安全扩展 · CodeQL & Supply Chain

```
name: CodeQL
on: [push, pull_request]
jobs:
 analyze:
    uses: github/codeql-action/.github/workflows/codeql.yml@v3
    with:
  languages: javascript, python
      queries: security-and-quality
```

Secret Scanning：扫描历史与增量提交；Push Protection 阻断含密钥的推送

© 版权所有 Microsoft Corporation。保留所有权利。

CodeQL 扫描机制与配置

安全扩展 · CodeQL & Supply Chain

- **分析原理**：CodeQL 把源代码编译成可查询数据库，用查询套件做语义 / 数据流分析，能发现注入类深层缺陷。
- **支持语言**：JavaScript/TypeScript、Python、Java、Go、C#、C/C++、Ruby 等；编译型语言需正确构建。
- **触发策略**：push 保证主干覆盖、pull_request 做增量门禁、schedule 定期全量重扫。
- **高级配置**：自定义 queries 查询套件、paths / paths-ignore 过滤、config-file 指定配置文件。
- **结果流转**：告警以 SARIF 上传至 code scanning，可在 PR 上注解并设为必填检查。
- **动手要点**：用 github/codeql-action/init → autobuild（或手动构建）→ analyze 三段式，或直接引用官方 CodeQL 工作流。

© 版权所有 Microsoft Corporation。保留所有权利。

Secret Scanning 与 Push Protection

安全扩展 · CodeQL & Supply Chain

- **检测范围**：扫描提交历史与新增提交中的凭据模式，覆盖大量服务商 token 格式与自定义模式。
- **Push Protection**：在推送时实时拦截命中规则的提交，把泄漏挡在仓库之外而非事后补救。
- **自定义模式**：企业内部 token 格式可定义自定义检测模式，并支持指定排除路径。
- **告警处置**：判定为真实泄漏时第一动作是轮换凭据，再关闭告警；误报标记后用于规则调优。
- **绕过审批**：必要时可配置允许带理由绕过，理由会进入审计日志，便于事后复核。
- **与流水线结合**：与 CodeQL、依赖扫描一并纳入合并门禁，形成三线安全网。

© 版权所有 Microsoft Corporation。保留所有权利。

依赖安全、SBOM 与供应链

安全扩展 · CodeQL & Supply Chain

- **Dependency graph**：依赖图是 Dependabot alerts 与依赖审查的基础，需确认仓库已启用。
- **Dependabot**：自动提交依赖升级 PR / 安全更新，也可用于升级 Actions 版本。
- **Dependency review**：在 PR 中检查新增依赖的已知漏洞与许可证，阻断有风险的引入。
- **SBOM**：导出软件物料清单（SPDX / CycloneDX），支撑合规与应急响应。
- **制品可信**：对构建产物签名并校验来源，与 Actions 版本固定形成完整链路。
- **企业落地**：用 Security Configurations 把上述能力批量下发到组织内全部仓库。

© 版权所有 Microsoft Corporation。保留所有权利。

Copilot 2026 新特性



© 版权所有 Microsoft Corporation。保留所有权利。

Copilot 2026：从补全到智能体

Copilot 2026 · Agentic Development

- GitHub Copilot 已从代码补全演进为智能体化开发平台
- 云 Coding Agent：在云端沙箱中自主完成需求分析、编码、测试、提交
- Agentic Review：自动审查 PR，给出修复建议与上下文说明
- 与 GitHub Actions 协同：把智能体嵌入流水线实现自动修复与摘要

- **云 Coding Agent：**
  - 基于 Issue 或 PR 触发，在隔离沙箱中运行，不影响主线
  - 自动克隆、分析代码、编写修改、运行测试、创建 PR
  - 典型场景：缺陷修复、特性实现、技术债务清理、文档补全

© 版权所有 Microsoft Corporation。保留所有权利。

Coding Agent 实操流程

Copilot 2026 · Agentic Development

- **触发**：把 Issue 指派给 Copilot 或在 PR 中发起请求，任务进入云端隔离沙箱执行。
- **环境**：沙箱内可配置依赖安装、构建与测试命令；网络与权限受企业策略约束。
- **执行**：智能体克隆仓库、分析上下文、修改代码、运行测试，并产出草稿 PR。
- **人工评审**：产出必须经人审阅：检查实现、测试覆盖、安全影响与是否符合规范。
- **适用场景**：边界清晰的小缺陷修复、依赖升级、测试补全、文档更新；不适合模糊的架构决策。
- **课堂建议**：把它定位为「加速器」而非「替代者」，强调评审与回滚始终是人的责任。

© 版权所有 Microsoft Corporation。保留所有权利。

智能体权限与风险管控

Copilot 2026 · Agentic Development

- **最小权限**：为智能体分配受限的仓库 / 组织权限，禁止直接推送生产分支。
- **AGENTS.md 规范**：用仓库内规范文件说明构建方式、代码风格、禁止修改的目录与敏感操作边界。
- **提交可信**：要求提交签名与可追溯的 PR 来源，把智能体产出纳入常规代码评审。
- **网络与数据**：限制沙箱出站访问，防止代码或凭据外发；明确数据驻留与合规要求。
- **审计与回退**：记录智能体触发、修改与审批链路，确保任何产出都能快速回滚。
- **治理归属**：智能体与人类开发者适用同一套 Rulesets、必填检查与安全门禁。

© 版权所有 Microsoft Corporation。保留所有权利。